

TP N°1: arVault

[75.73/TB034] Arquitectura de Software

CURSO: 01-Calónico

Primer cuatrimestre de 2026

Alumno/a:	FARALL CASADO, Tomás
Número de padrón:	102628
Email:	tcada@fi.uba.ar

Alumno/a:	VACCARELLI, Santiago
Número de padrón:	106051
Email:	svaccarelli@fi.uba.ar

Alumno/a:	JANG, Lucas
Número de padrón:	109151
Email:	ljang@fi.uba.ar

Alumno/a:	KRESTA, Facundo Ariel
Número de padrón:	110857
Email:	facundoarielkresta@gmail.com

7 de mayo de 2026

Índice

1	Introducción	4
1.1	Contexto	4
1.2	Objetivo	4
1.3	Stack tecnológico	4
1.4	Topología de despliegue	4
1.5	Patrón arquitectónico	5
1.6	Endpoints disponibles	5
2	Atributos Importantes	6
2.1	Atributos de calidad clave	6
2.1.1	Seguridad (Security):	6
2.1.2	Disponibilidad (Availability):	6
2.1.3	Desempeño (Performance):	6
2.1.4	Escalabilidad (Scalability):	6
2.1.5	Usabilidad (Usability):	7
2.2	Otros atributos de calidad	7
2.2.1	Visibilidad / Observabilidad (Visibility)	7
2.2.2	Testabilidad (Testability)	7
2.2.3	Portabilidad (Portability)	7
2.2.4	Interoperabilidad (Interoperability)	7
3	Decisiones de Diseño (C&C)	8
3.1	Evolución: Del Monolito Acoplado a Arquitectura en Capas	8
3.1.1	Mejora propuesta	8
3.1.2	Atributos de calidad afectados	9
3.1.2.1	Testability	9
3.1.2.2	Maintainability	10
3.1.2.3	Modifiability	10
3.1.2.4	Performance (atributo afectado negativamente)	10
3.2	Variables en memoria	10
3.2.1	Mejora propuesta	10
3.2.2	Atributos de calidad afectados	12
3.2.2.1	Scalability	12
3.2.2.2	Performance (atributo afectado negativamente)	12
3.3	Persistencia lazy	12
3.3.1	Mejora propuesta	12
3.3.2	Atributos de calidad afectados	12
3.3.2.1	Durability	12
3.3.2.2	Performance	12
3.4	Persistencia en archivos JSON	12
3.4.1	Mejora propuesta	13
3.4.2	Atributos de calidad afectados	13
3.4.2.1	Availability	13
3.4.2.2	Reliability	13
3.4.2.3	Efficiency	13
3.4.2.4	Scalability	13
3.4.2.5	Testability	13
3.5	Función <code>transfer()</code> mockeada de forma secuencial	13
3.6	Instancia única sin redundancia ni healthcheck	14
3.6.1	Modos de falla que quedan silenciosos sin healthcheck	14
3.6.2	Blast radius: cualquier falla = todos los usuarios afectados	15
3.6.3	Configuración de Nginx: mala incluso asumiendo instancia única	15

3.6.4	Docker Compose: lo que falta en la declaración del servicio	16
3.6.5	Mejoras propuestas	16
3.6.6	Resumen de impacto sobre QA	18
3.7	HTTP 500 para errores de negocio	18
3.7.1	Atributos de calidad afectados	18
3.7.1.1	Visibility	18
3.7.1.2	Usabilidad	18
3.8	Log de transacciones ilimitado en memoria	19
3.8.1	Mejora propuesta	19
3.8.2	Atributos de calidad afectados	19
3.8.2.1	Network Performance	19
3.8.2.2	Efficiency	19
3.8.2.3	Availability	19
3.8.2.4	Usabilidad	19
3.9	Impacto de las Decisiones de Diseño sobre los QA	19
3.9.1	Sistema original	19
3.9.2	Luego de las mejoras propuestas	20
4	Pruebas	22
4.0.1	Correr los escenarios	22
4.0.1.1	Escenarios disponibles	22
4.0.1.2	Workloads disponibles	22
4.1	Objetivo	22
4.2	Workload Models (Modelos de Carga)	22
4.2.1	Baseline (Smoke / Low-Constant)	22
4.2.2	Load Test (Carga Normal Sostenida)	23
4.2.3	Stress Test (Buscar el Punto de Quiebre)	23
4.3	Escenarios de Artillery a Implementar	23
4.3.1	perf/rates.yaml (dado por la cátedra)	23
4.3.2	perf/exchange.yaml (nuevo)	23
4.3.3	perf/mixed.yaml (nuevo)	23
4.4	Plan de Ejecución	24
4.4.1	Fase A – Baseline	24
4.4.2	Fase B – Redis + arquitectura de capas + healthcheck + keepalive	24
4.4.3	Fase C – Escalado horizontal (3 instancias)	24
4.5	Análisis de resultados	24
4.5.1	Escenario ‘rates‘ (GET /rates)	24
4.5.1.1	Baseline	24
4.5.1.2	Load	25
4.5.1.3	Stress	26
4.5.1.4	Conclusión	27
4.5.2	Escenario ‘exchange‘ (POST /exchange)	28
4.5.2.1	Baseline	28
4.5.2.2	Load	29
4.5.2.3	Stress	30
4.5.2.4	Conclusión	31
4.5.3	Escenario ‘mixed‘ (mix 60 % rates / 30 % exchange / 5 % accounts / 5 % log)	31
4.5.3.1	Baseline	31
4.5.3.2	Load	32
4.5.3.3	Stress	33
4.5.3.4	Conclusión	34
4.6	Puntos De Ruptura Identificados	34
4.6.1	‘/rates‘ (Read-Only)	34
4.6.2	‘/exchange‘ (Business Logic)	34

4.6.3	‘/mixed’	34
4.7	Análisis Profundo: ¿Por Qué Falla?	34
4.7.1	Latencia Base de ‘/exchange’	34
4.7.2	Single-Threaded Event Loop	35
4.7.3	Race Condition	35
4.8	Fase B (Redis + arquitectura de capas + healthcheck + keepalive)	35
4.8.1	Resumen rápido	35
4.8.2	Análisis: ¿Por qué errores en mixed y por qué exchange es cuello de botella?	35
4.8.3	Respuestas por escenario (Fase B)	36
4.8.3.1	Escenario rates	36
4.8.3.2	Escenario exchange	36
4.8.3.3	Escenario mixed	37
4.9	Fase C – Escalado horizontal (3 instancias)	38
4.9.1	Archivos generados	38
4.9.2	Resumen ejecutivo	38
4.9.3	Respuestas por escenario (Fase C)	39
4.9.3.1	exchange – Load	39
4.9.3.2	exchange – Stress	40
4.9.3.3	mixed – Load	41
4.9.3.4	mixed – Stress	42
4.9.4	Comparación con Fase B	43
4.9.5	Análisis: ¿Por qué desaparecieron los errores 5xx/ETIMEDOUT?	43
4.9.6	Hallazgos clave (evidencia)	43
4.10	Conclusión – Comparativa de las 3 fases	43
4.10.1	Qué se midió	43
4.10.2	Fase A – Baseline (proyecto original)	43
4.10.3	Fase B – Redis + arquitectura de capas + healthcheck + keepalive	44
4.10.4	Fase C – Escalado horizontal (3 instancias)	44
4.10.5	Conclusión general	44
4.11	Métricas de Negocio: Volumen Operado y Neto por Moneda	44
4.11.1	Implementación	44
4.11.2	Métricas emitidas	45
4.11.3	Interpretación	45
4.11.4	Evidencia visual	45
4.11.5	Observaciones	46
4.11.6	Impacto en los atributos de calidad	46

5 Conclusión

47

1. Introducción

1.1. Contexto

La startup **arVault** es una fintech que opera una billetera digital, de reciente creación. Su fundador, un desarrollador aficionado y entusiasta con algo de dinero y muchas ideas, consciente de que la clave de su negocio es la implementación de su **backend**, tercerizó el desarrollo de las aplicaciones para dispositivos móviles, y se concentró en implementar rápidamente (bajo la consigna *fake it till you make it*¹) un núcleo de servicios que le permitieran conseguir más fondos a través de inversores.

Luego de la ronda inicial, los primeros fondos fueron utilizados, no para robustecer la arquitectura existente, sino para agregar más funcionalidad. Una de estas funcionalidades nuevas permite abrir cuentas en distintas monedas (que se respaldan en bancos existentes), y realizar operaciones de cambio entre éstas. El diferencial de **arVault** es tener la tasa de cambio más conveniente dentro de las aplicaciones que proveen este servicio. Para ganar usuarios, las tasas de cambio no tienen *gap* entre el valor de compra y de venta.

Si bien el lanzamiento fue exitoso, comenzaron a aparecer reclamos de los usuarios sobre problemas en el uso de la función de cambio de monedas. Estos reclamos llegaron en el peor momento, dado que **arVault** necesita captar más inversiones y, debido a la caída de la reputación y las reviews negativas, los potenciales inversores se niegan a aportar más fondos si no se realiza una auditoría y se mejora el servicio.

Frente a este reclamo, **arVault** decide contratar a un grupo de arquitectos para que evalúen, propongan e implementen soluciones que mejoren los atributos de calidad del servicio de cambio de monedas.

1.2. Objetivo

El objetivo de este informe es presentar una evaluación de la arquitectura actual del servicio de cambio de monedas, identificar los atributos de calidad que se ven afectados por los reclamos de los usuarios, y proponer decisiones de diseño que puedan mejorar dichos atributos. Además, se presentarán pruebas y métricas en el para evaluar el impacto de las decisiones propuestas y se discutirán las conclusiones obtenidas a partir del análisis realizado. Finalmente, se incluirá una sección de conclusiones que sintetice los hallazgos y recomendaciones para **arVault**.

1.3. Stack tecnológico

Componente	Tecnología	Versión
API REST	Node.js + Express	22 / 4.21.2
Persistencia	Archivos JSON en disco	—
Contenedores	Docker + Docker Compose	—
Reverse proxy	Nginx	1.29.8
Generación de carga	Artillery + plugin-statsd	2.0.22 / 2.2.1
Métricas	StatsD + Graphite + Grafana	— / 1.1.10-5 / 11.5.10
Monitoreo de contenedores	cAdvisor	0.51.0

1.4. Topología de despliegue

```

Cliente / Artillery (load generator)
  |
  v
+-----+
|  Nginx  | <- Reverse proxy, puerto 5555->80
| (proxy) |   upstream hardcoded: exchange-api-1:3000
+-----+

```

¹Expresión popular que refiere a simular confianza o habilidades hasta realmente adquirirlas.

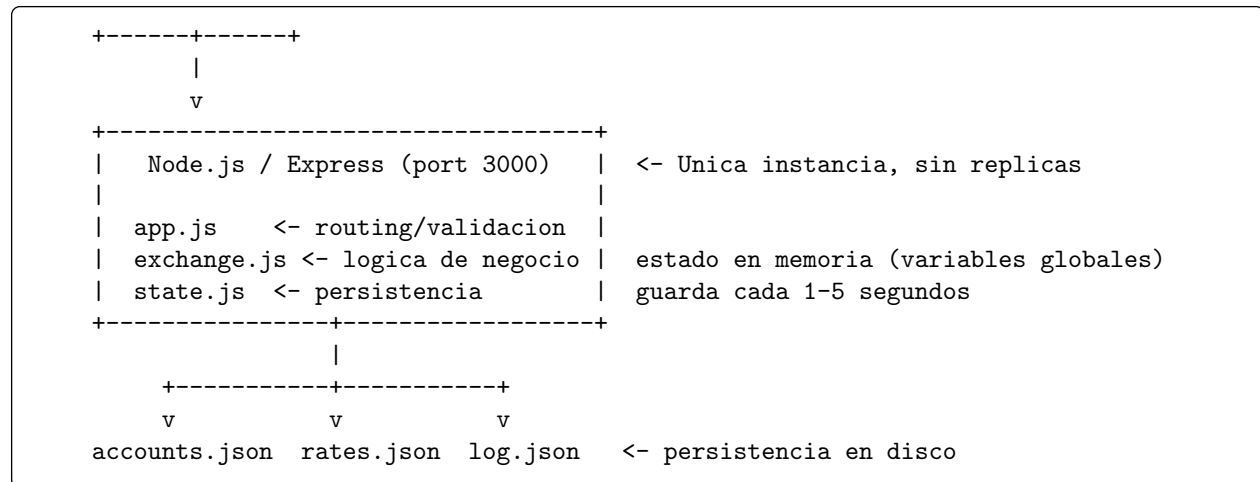


Diagrama 1.1: Arquitectura primera del servicio.

Stack de observabilidad (desacoplado de la app):

```

Artillery --> StatsD --> Graphite <-- cAdvisor
                        |
                        Grafana
(4 paneles: RPS, codigos HTTP,
latencia cliente, CPU/memoria)

```

1.5. Patrón arquitectónico

Monolito de **tres capas** (controller → business logic → persistence) con **estado centralizado en memoria** y **persistencia eventual** en archivos JSON. No hay base de datos relacional, no hay cola de mensajes, no hay mecanismo de coordinación distribuida.

1.6. Endpoints disponibles

Método	Ruta	Descripción
GET	/accounts	Retorna cuentas internas de la empresa
PUT	/accounts/:id/balance	Modifica el saldo de una cuenta interna
GET	/rates	Retorna tasas de cambio vigentes
PUT	/rates	Actualiza una tasa y calcula la recíproca
POST	/exchange	Ejecuta una operación de cambio de monedas
GET	/log	Retorna el historial de transacciones

2. Atributos Importantes

2.1. Atributos de calidad clave

Considerando la naturaleza financiera y transaccional del servicio provisto (*arVault*, un *exchange* de criptomonedas y dinero fiat), se han determinado como centrales los siguientes Atributos de Calidad (QAs):

2.1.1. Seguridad (Security):

Al tratarse de una *fintech* que opera saldos reales en múltiples monedas, la seguridad es el atributo más crítico. Si bien existe un componente externo que gestiona autenticación y autorización, hay aspectos de seguridad que recaen directamente sobre el servicio: la integridad de las transacciones, la prevención de condiciones de carrera que permitan operar con fondos insuficientes y la consistencia del estado financiero. Una brecha en este atributo tiene consecuencias legales, financieras y reputacionales inmediatas. El servicio expone operaciones que modifican saldos reales y tasas de cambio que afectan a todos los usuarios. Sin autenticación ni autorización propias, cualquier actor con acceso de red puede manipular el estado financiero del sistema. Para los inversores que evalúan arVault, una brecha de seguridad evidente es un bloqueante.

2.1.2. Disponibilidad (Availability):

arVault ya sufrió caída de reputación y pérdida de inversores por problemas en el servicio de cambio. Para una billetera digital, la indisponibilidad equivale a pérdida de negocio y confianza de los usuarios. Este atributo es condición necesaria para la continuidad operativa de la empresa. El servicio ya está en producción con usuarios activos. Un tiempo de inactividad tiene impacto directo en reputación, en las reviews negativas que motivan este análisis y en la confianza de los potenciales inversores. En *fintech*, los usuarios asocian la indisponibilidad del servicio con pérdida de dinero o acceso bloqueado a sus fondos, lo que amplifica el daño reputacional.

2.1.3. Desempeño (Performance):

El diferencial de arVault es ofrecer la tasa de cambio más conveniente del mercado. Si las operaciones de *exchange* presentan latencias elevadas, los usuarios perciben el servicio como poco confiable, especialmente en un contexto donde los tipos de cambio fluctúan en tiempo real. El sistema debe procesar múltiples operaciones concurrentes con un tiempo de respuesta adecuado y un alto nivel de *throughput* bajo momentos de mayor demanda. Performance se descompone en tres dimensiones diferenciadas para este servicio:

- **2a — Network Performance:** Eficiencia en el uso de la red entre cliente y servidor (tamaño de respuestas, reutilización de conexiones, compresión).
- **2b — User-Perceived Performance:** Latencia observable por el usuario final desde que emite un request hasta que recibe respuesta. Es el QA de performance más crítico para la experiencia de usuario: determina si arVault se percibe como “rápida” o “lenta”.
- **2c — Efficiency (Resource Efficiency):** Uso eficiente de CPU, memoria y I/O por parte del sistema. Impacta el costo de operación y la degradación del servicio bajo carga sostenida.

El diferencial de arVault es la mejor tasa de cambio del mercado. Si la experiencia es lenta, los usuarios prefieren competidores con tasas levemente peores pero mejor UX. Además, bajo carga concurrente, la ineficiencia del servicio puede amplificar otros problemas (race conditions, I/O blocking).

2.1.4. Escalabilidad (Scalability):

La startup se encuentra en crecimiento activo, captando usuarios e inversores. La arquitectura original fue construida bajo la consigna *fake it until you make it* y no fue diseñada para escalar. Previendo un escenario con cantidad creciente de usuarios operando simultáneamente, el servicio debe poder ser replicado horizontalmente para poder soportar la carga aumentada de usuarios. Si arVault capta inversión y crece en usuarios, la arquitectura debe soportar mayor carga. La escalabilidad determina si el sistema puede crecer

horizontalmente (más instancias) o solo verticalmente (máquina más grande). En un contexto de startup, la escalabilidad horizontal es más económica y resiliente.

2.1.5. Usabilidad (Usability):

El fundador de arVault tercerizó el desarrollo de las aplicaciones móviles, por lo que la usabilidad del frontend queda fuera del alcance directo de este análisis. Se menciona por su importancia, pero no es objetivo de las tácticas a implementar en este trabajo. Los consumidores directos de este servicio son las apps mobile desarrolladas externamente. Una API con semántica incorrecta, mensajes de error crípticos o comportamiento inconsistente obliga a los desarrolladores de las apps a escribir código defensivo complejo y dificulta el diagnóstico de problemas en el cliente.

2.2. Otros atributos de calidad

Además de los QAs clave, se identifican otros atributos que, aunque no son el foco principal de este trabajo, también son relevantes para la calidad general del sistema:

2.2.1. Visibilidad / Observabilidad (Visibility)

Para diagnosticar los reclamos de usuarios y operar un servicio financiero con confianza, el equipo necesita saber en tiempo real qué está pasando dentro del sistema: tasas de error, latencias, volúmenes, estado de los saldos. Sin visibilidad, los problemas se descubren cuando los usuarios se quejan, no cuando ocurren.

2.2.2. Testeabilidad (Testability)

Sin tests automatizados, cada cambio en el código puede introducir regresiones que solo se descubren en producción. En un servicio que maneja dinero, la falta de testeabilidad es un multiplicador de riesgo: no se puede verificar el comportamiento antes de desplegar.

2.2.3. Portabilidad (Portability)

La capacidad de desplegar el sistema en distintos entornos (desarrollo, staging, producción, CI/CD) sin modificar el código reduce la fricción operativa y previene el síndrome de “funciona en mi máquina”. En una startup que itera rápido, esto es crítico para la velocidad de desarrollo.

2.2.4. Interoperabilidad (Interoperability)

El servicio de cambio no opera en aislamiento: depende de un servicio de transferencias externo (actualmente mockeado), necesita integrarse con vaultSec para autenticación, y eventualmente deberá conectarse con bancos reales. La interoperabilidad determina qué tan fácil es integrar, reemplazar o extender estas dependencias.

3. Decisiones de Diseño (C&C)

3.1. Evolución: Del Monolito Acoplado a Arquitectura en Capas

Originalmente, el servicio constaba de un monolito de una sola capa. Había tres archivos con roles distintos (`app.js` operando de HTTP handler, `exchange.js` manejando lógica de negocio, y `state.js` interactuando con la persistencia en disco) que se encontraban acoplados directamente sin una clara inyección de dependencias ni definición de interfaces seguras. Todo corría en el hilo de un único proceso de Node.js, penalizando *Testability*, *Maintainability* y *Modifiability*. El caso base se ilustra debajo:

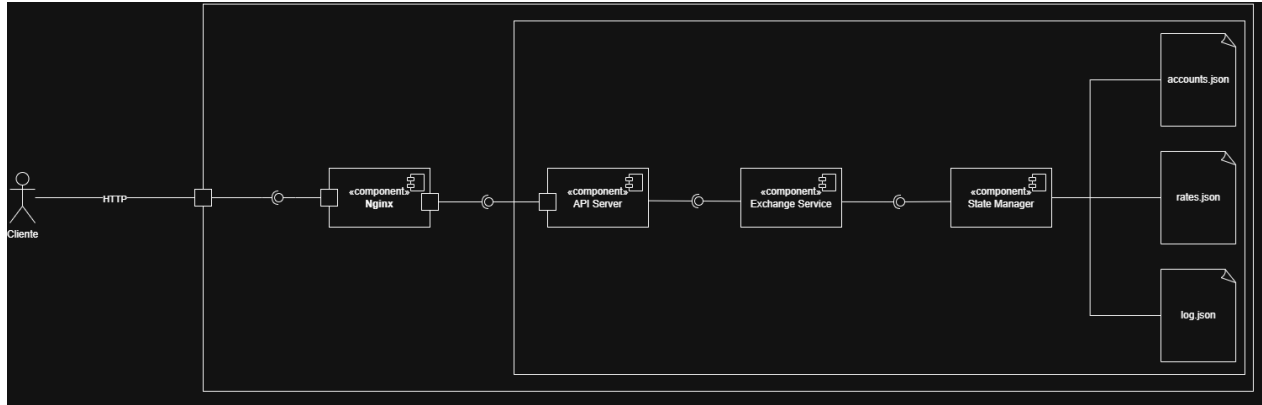


Diagrama 3.1: Arquitectura monolítica del sistema

El **Exchange Service** accede directamente a referencias internas del **State Manager** y muta su estado sin pasar por ninguna interfaz definida, *bypaseando* por completo los métodos expuestos por el **State Manager**.

Además, el **API Server** mezcla responsabilidades: realiza validación de input, invoca la lógica de negocio y decide el formato de respuesta todo en el mismo handler, sin delegar al **Exchange Service** de forma limpia.

3.1.1. Mejora propuesta

La arquitectura propuesta introduce una separación en capas con responsabilidades bien delimitadas. El **API Server** delega cada request a uno de cuatro controladores especializados: **Account Controller**, **Rate Controller**, **Exchange Controller** y **Log Controller**, que se encargan exclusivamente de la validación de input y el mapeo de respuestas HTTP. Cada controlador invoca a su correspondiente servicio de dominio (**Account Service**, **Rate Service**, **Exchange Service** y **Log Service**), donde reside la lógica de negocio. Los servicios acceden al estado únicamente a través de repositorios (**Account Repository**, **Rate Repository**, **Log Repository**), que exponen interfaces limpias y son los únicos componentes autorizados a interactuar con el **State Manager**. Este último centraliza toda la lectura y escritura a disco, persistiendo el estado en `accounts.json`, `rates.json` y `log.json`.

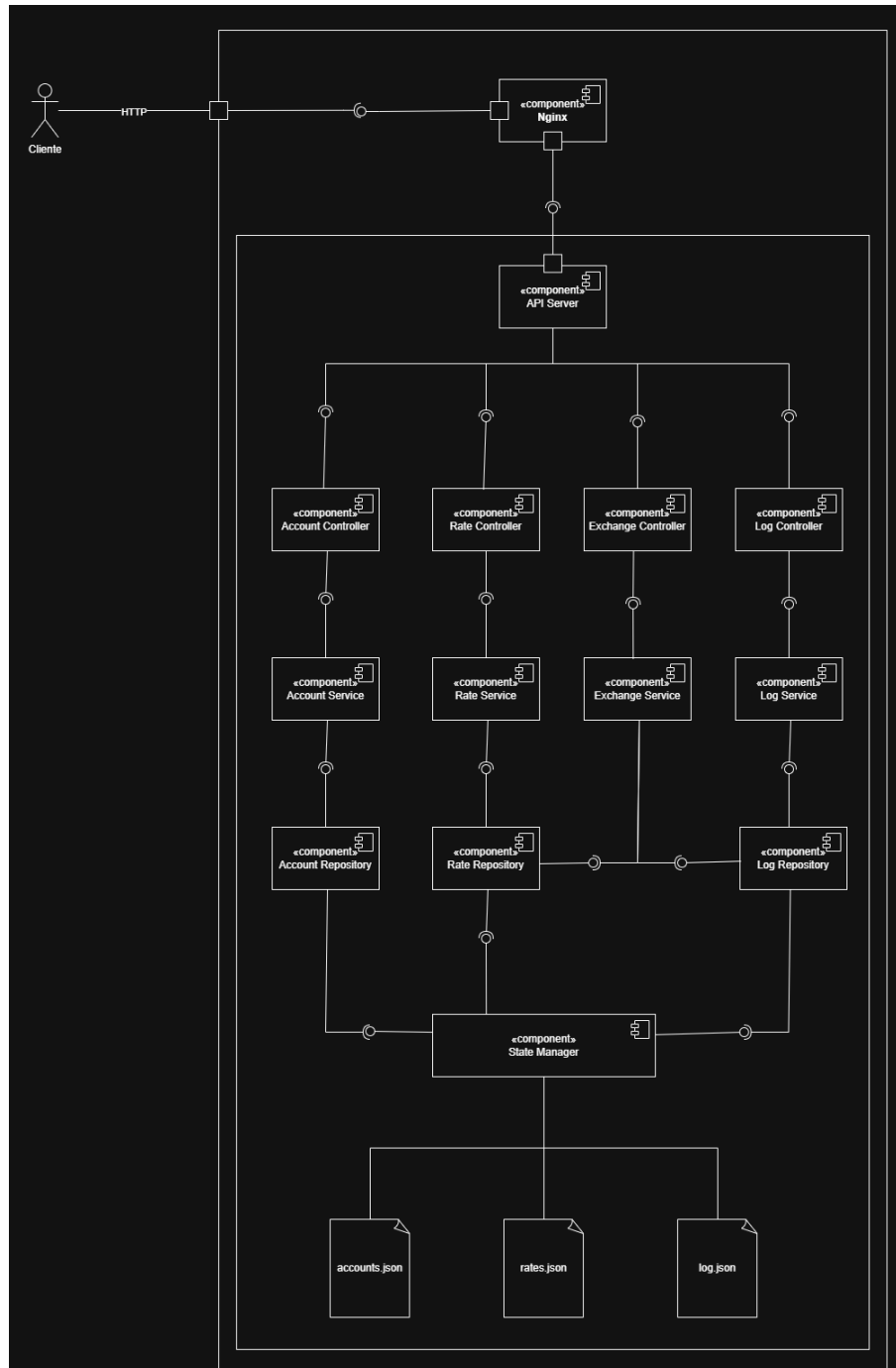


Diagrama 3.2: Arquitectura luego de la mejora

3.1.2. Atributos de calidad afectados

3.1.2.1 Testability

Es el atributo más beneficiado por este cambio. En la arquitectura original, testear el **Exchange Service** implicaba activar toda la cadena de componentes, ya que no había separación entre lógica de negocio, acceso a datos y manejo de estado. En la nueva arquitectura, cada capa puede probarse de forma aislada: los Controllers pueden testearse con mocks de los Services, los Services con mocks de los Repositories, y los Repositories con mocks del **State Manager**. Esto permite pruebas unitarias verdaderas y facilita la detección temprana de

bugs.

3.1.2.2 Maintainability

También mejora considerablemente. La separación por capas y por responsabilidades hace que encontrar a nivel de código dónde está cada cosa sea más fácil e intuitivo, sin tener que recorrer código ajeno a lo que se busca. Si surge un bug, el patrón establecido lo hace directo de encontrar; cada feature nuevo sigue la misma estructura.

3.1.2.3 Modifiability

El cambio tiene un impacto positivo significativo. Con la arquitectura anterior, agregar un nuevo dominio o modificar uno existente podía requerir alterar componentes compartidos como el **Exchange Service**, propagando el riesgo de cambio por toda la cadena. En la nueva arquitectura, cada dominio es una columna vertical independiente (**Controller** → **Service** → **Repository**), de modo que agregar un nuevo dominio implica simplemente agregar una nueva columna sin tocar las existentes. Del mismo modo, cambiar la fuente de datos de los logs, por ejemplo, solo requiere modificar el **Log Repository**, sin afectar al **Log Service** ni al **Log Controller**. Esto reduce el costo y el riesgo de cada modificación futura.

3.1.2.4 Performance (atributo afectado negativamente)

Si bien en teoría la nueva arquitectura introduce más niveles de indirección y por lo tanto podría esperarse un impacto negativo en la latencia, en la práctica las métricas no mostraron diferencias significativas respecto a la arquitectura anterior, o las diferencias registradas fueron lo suficientemente pequeñas como para considerarse despreciables. El overhead generado por los saltos adicionales entre capas no se tradujo en una degradación observable del sistema.

3.2. Variables en memoria

Accounts, rates y log son variables JavaScript dentro del proceso. Leer es instantáneo (bueno para *performance*), pero hace imposible escalar horizontalmente porque cada instancia tendría su propio estado desincronizado.

3.2.1. Mejora propuesta

Se incorporó Redis como almacenamiento centralizado. El estado deja de vivir dentro del proceso y pasa a una fuente de verdad única y compartida entre todas las instancias, eliminando la desincronización.

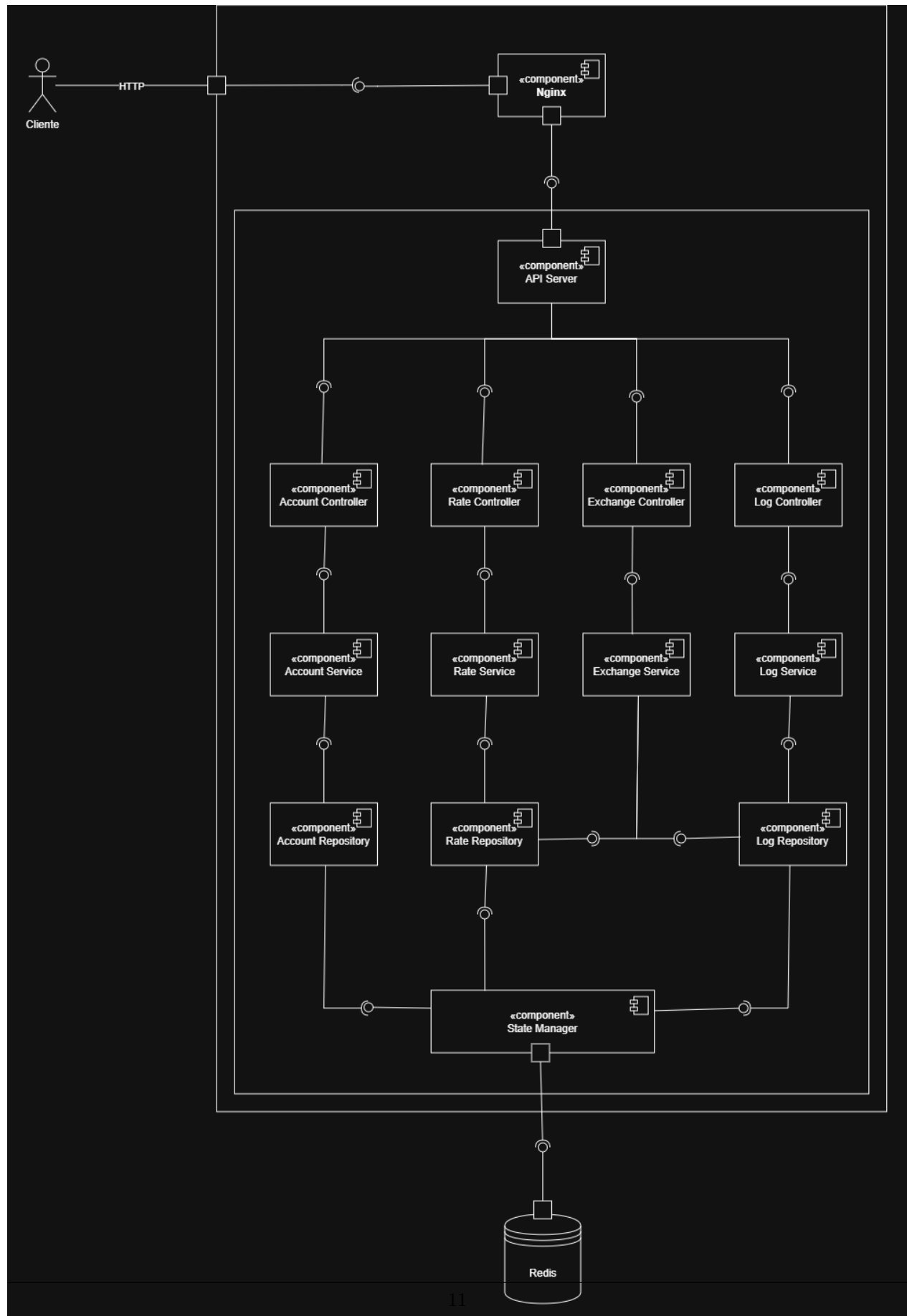


Diagrama 3.3: Arquitectura con Redis como almacenamiento centralizado. Las tres decisiones de diseño que siguen —variables en memoria, persistencia lazy y persistencia en archivos JSON— fueron resueltas todas juntas mediante la incorporación de Redis. El **State Manager** ya no persiste en archivos locales sino que delega en Redis, que actúa como fuente de verdad única y compartida entre todas las instancias.

3.2.2. Atributos de calidad afectados

3.2.2.1 Scalability

El atributo más impactado. Antes era imposible levantar múltiples instancias porque cada una tendría su propio estado desincronizado; con Redis esto queda resuelto.

3.2.2.2 Performance (atributo afectado negativamente)

Leer una variable en memoria es instantáneo, mientras que cada operación contra Redis implica una llamada de red. En la práctica esta latencia suele ser de un par de milisegundos y resulta aceptable, pero es un costo real que antes no existía. En nuestras pruebas la diferencia en performance era despreciable, por lo que el trade-off entre escalabilidad y latencia es favorable.

3.3. Persistencia lazy

El estado se guarda a disco cada 1-5 segundos en lugar de en cada operación. Simplifica el código y no bloquea requests con I/O, pero si el proceso muere entre guardados se pierden transacciones, violando la durabilidad.

3.3.1. Mejora propuesta

Con la incorporación de Redis se removió la persistencia lazy a disco. Redis se encarga de la durabilidad y persistencia de los datos, eliminando la ventana de pérdida de transacciones en caso de caída del proceso.

3.3.2. Atributos de calidad afectados

3.3.2.1 Durability

La ventana de pérdida de datos que generaba el guardado cada 1-5 segundos desaparece. Si el proceso muere, las transacciones están garantizadas porque Redis las persistió antes de confirmarlas.

3.3.2.2 Performance

Dependiendo del nivel de durabilidad configurado en Redis, puede agregarse algo de latencia por operación. Es un costo generalmente aceptable frente a la garantía que ofrece, pero es un trade-off a tener en cuenta.

3.4. Persistencia en archivos JSON

Archivos: `app/state.js:16--23`, `app/state.js:55--68`

```
// Carga inicial desde disco
accounts = await load(ACCOUNTS); // state.js:17
rates    = await load(RATES);    // state.js:18
log      = await load(LOG);      // state.js:19

// Guardado periódico asíncrono
scheduleSave(accounts, ACCOUNTS, 1000); // cada 1 segundo
scheduleSave(rates,    RATES,    5000); // cada 5 segundos
scheduleSave(log,      LOG,      1000); // cada 1 segundo
```

Todo el estado vive en archivos JSON locales. No hay escritura atómica (se usa `writeFile` directamente, sin `write-to-temp-then-rename`), no hay transacciones atómicas, el array `log` crece ilimitadamente (`exchange.js:108`) y es imposible compartir estado entre instancias.

3.4.1. Mejora propuesta

Con la incorporación de Redis se reemplazó completamente la persistencia en archivos JSON por un almacenamiento centralizado. Esto resuelve la falta de transacciones atómicas, la imposibilidad de compartir estado entre instancias y el riesgo de corrupción.

3.4.2. Atributos de calidad afectados

3.4.2.1 Availability

Un crash durante `writeFile` (por ejemplo, disco lleno o SIGKILL) dejaba el archivo JSON parcialmente escrito. Al reiniciar, `JSON.parse` lanzaba una excepción en `state.js:45` y el servicio quedaba inutilizable hasta intervención manual. Con Redis este riesgo desaparece, aunque se introduce una nueva dependencia de infraestructura: si Redis no está disponible, el sistema tampoco funciona.

3.4.2.2 Reliability

El riesgo de corrupción de archivos ya no existe. Redis maneja la integridad de los datos de forma mucho más robusta que un archivo JSON que puede quedar en estado inconsistente si el proceso muere en medio de una escritura. Adicionalmente, Redis permite operaciones atómicas, eliminando la *race condition* TOCTOU del doble gasto donde dos requests concurrentes podían leer el mismo balance y pasar ambas el check de fondos simultáneamente.

3.4.2.3 Efficiency

El array `log` crecía ilimitadamente y `scheduleSave` serializaba el array completo cada 1 segundo mediante `JSON.stringify`. Con alta carga sostenida, esta operación escalaba linealmente en CPU y tiempo, compitiendo con el procesamiento de requests. Redis elimina este problema.

3.4.2.4 Scalability

Era arquitecturalmente imposible correr más de una réplica simultánea de la API: dos procesos escribiendo el mismo `accounts.json` en simultáneo producirían corrupción sin ningún mecanismo de coordinación. Con Redis como fuente de verdad es posible escalar horizontalmente agregando mas instancias de la API, todas compartiendo el mismo estado.

3.4.2.5 Testability

Los tests de integración debían preparar y limpiar archivos JSON en disco. Con Redis es posible inyectar estado sin tocar el filesystem, simplificando el setup de tests.

3.5. Función `transfer()` mockeada de forma secuencial

Archivo: `app/exchange.js:114{120`

```
async function transfer(fromAccountId, toAccountId, amount) {
  const min = 200;
  const max = 400;
  return new Promise((resolve) =>
    setTimeout(() => resolve(true), Math.random() * (max - min + 1) + min));
}
```

La función ignora todos sus parámetros y siempre resuelve `true` tras un delay aleatorio de 200–400 ms. Las dos llamadas en `exchange()` son **secuenciales** (líneas 83 y 86: la segunda `await` comienza solo cuando la primera termina).

3.6. Instancia única sin redundancia ni healthcheck

Archivos: `docker-compose.yml` (completo), `nginx_reverse_proxy.conf`

```
# nginx_reverse_proxy.conf
upstream api {
    server exchange-api-1:3000;    ← único upstream, hardcodedo
}
server {
    listen 80;
    location / {
        proxy_pass http://api/;
        # sin timeouts, sin retry, sin health, sin keepalive
    }
}

# docker-compose.yml | sin réplicas, sin healthcheck, sin restart policy
services:
  api:
    build: ./app
    # sin deploy.replicas
    # sin healthcheck
    # sin restart: unless-stopped
    # sin límite de memoria
    # sin límite de CPU
```

Esta es una de las decisiones de diseño con más superficie de impacto. A continuación se detallan los modos de falla específicos, el blast radius y los QA afectados.

3.6.1. Modos de falla que quedan silenciosos sin healthcheck

Un healthcheck distingue entre “proceso corriendo” y “proceso respondiendo correctamente”. Sin él, Docker y Nginx solo saben si el PID 1 del contenedor está vivo. Los siguientes modos de falla dejan el contenedor en estado “running” pero el servicio efectivamente caído:

1. **Event loop bloqueado por operación síncrona.** Node.js es single-threaded. Una operación síncrona costosa (e.g., `JSON.stringify` sobre un log de millones de entradas en `state.js:58`) congela el procesamiento de requests durante segundos. Docker reporta el contenedor “running” durante todo ese tiempo; Nginx sigue enviando requests que se encolan en el socket TCP hasta timeout.
2. **GC thrashing por crecimiento del heap.** El array log crece sin límite (`exchange.js:108`). Antes de que V8 lance OOM, entra en ciclos de garbage collection cada vez más largos y frecuentes (“GC pause”), degradando la latencia sin crashear. Este estado puede durar minutos antes del crash terminal.
3. **Fallo silencioso de scheduleSave.** Los `setInterval` en `state.js:21{23}` ejecutan `save()` sin manejo de error en caso de que `writeFile` falle repetidamente (disco lleno, permisos, corrupción del FS). El error se loguea a `console.error` (`state.js:60`) y el intervalo continúa reintentando. El estado en memoria diverge del estado en disco sin que nadie se entere.
4. **Archivo JSON corrupto tras crash.** `state.js:58` usa `writeFile` directo (no atómico). Un crash durante la escritura deja el archivo con JSON parcial. En el próximo restart, `load()` en `state.js:45` lanza en `JSON.parse`, el catch en línea 46 loguea el error y retorna `undefined`. El servicio queda con `accounts = undefined` pero “running”. El primer request a `/exchange` produce un `TypeError` no capturado.
5. **Inicialización larga.** Al arrancar, `app.js:13` ejecuta `await exchangeInit()` que a su vez lee los tres archivos JSON secuencialmente (`state.js:17{19}`). Con un log.json grande, esto puede tomar varios segundos. Durante ese intervalo el puerto 3000 no escucha aún. Nginx, sin healthcheck, intenta proxear requests al upstream y obtiene `connection refused` → 502.

3.6.2. Blast radius: cualquier falla = todos los usuarios afectados

Al existir una única instancia, la falla no puede ser parcial:

- No hay **isolation entre usuarios**: un usuario que dispare un input que causa `TypeError` (sección 3.8) crashea el proceso y afecta a todos.
- No hay **isolation entre endpoints**: `GET /log` serializa todo el log y bloquea el event loop → `POST /exchange` de otro usuario se queda esperando.
- No hay **circuit breaker**: si el servicio externo de transferencias (cuando se integre real en reemplazo del mock de `exchange.js:114`) se degrada, todos los `POST /exchange` se encolan en el event loop aumentando la memoria y eventualmente tumbando el proceso.
- No hay **bulkhead**: los requests de lectura (`GET /rates`, relativamente baratos) comparten el mismo event loop que los requests pesados (`POST /exchange` con sus 400–800 ms). Bajo carga del segundo, los primeros se degradan igual.

3.6.3. Configuración de Nginx: mala incluso asumiendo instancia única

El bloque `upstream` tiene carencias que amplifican el problema aún con una sola instancia:

lo que falta en `nginx_reverse_proxy.conf`:

```
upstream api {
    server exchange-api-1:3000
        # ausente: Nginx no marca el upstream como down
        max_fails=3 fail_timeout=30s;
    # ausente: cada request abre conexión TCP nueva al upstream
    keepalive 32;
}

server {
    listen 80;
    location / {
        proxy_pass http://api/;
        # ausente: si el upstream cuelga, el cliente espera el default (60s)
        proxy_connect_timeout 5s;
        # ausente
        proxy_read_timeout 10s;
        # ausente: no hay retry hacia otro upstream (no hay otro, pero tampoco lógica de reintento)
        proxy_next_upstream error timeout;

        # ausente: necesario para keepalive con el upstream
        proxy_http_version 1.1;
        # ausente: necesario para keepalive
        proxy_set_header Connection "";

        # Sin headers informativos al upstream:
        # proxy_set_header X-Real-IP $remote_addr;
        # proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Consecuencias concretas:

- Sin `keepalive`, cada request entrante a Nginx abre y cierra una conexión TCP nueva contra el contenedor de la API. En una prueba de carga sostenida, esto satura la tabla de sockets (`TIME_WAIT`) del kernel mucho antes que la app realmente se quiebre.
- Sin `proxy_read_timeout`, un request `POST /exchange` que quede colgado por un `await transfer()` que nunca resuelve (en una integración real con un banco lento) ocupa un worker de Nginx hasta el default de 60 segundos.
- Sin `proxy_next_upstream` ni backends alternativos, cualquier error del upstream es un 502 al cliente sin mitigación.
- Sin `X-Forwarded-For`, el log de aplicación (si existiera uno real) no puede identificar la IP original del cliente.

3.6.4. Docker Compose: lo que falta en la declaración del servicio

lo que debería tener `services.api` en `docker-compose.yml`:

```
api:
  build: ./app
  # ausente: Docker no reinicia el contenedor si muere
  restart: unless-stopped
  deploy:
    ausente: permitiría escalar horizontalmente
    replicas: 3
    resources:
      limits:
        # ausente: sin límite, el contenedor puede agotar la RAM del host
        memory: 512M
        # ausente
        cpus: "1.0"
  # ausente completamente
  healthcheck:
    test: ["CMD", "wget", "-q", "-O-", "http://localhost:3000/health"]
    interval: 10s
    timeout: 3s
    retries: 3
    start_period: 15s
```

Consecuencias concretas:

- Sin `restart: unless-stopped`, un crash del proceso Node.js deja el contenedor en estado “exited” hasta intervención manual. El sistema queda caído indefinidamente.
- Sin endpoint de health en la aplicación (`/health` o `/ready`), no hay manera de implementar el healthcheck de Docker aunque se quisiera. El servicio debe exponer un endpoint trivial que verifique al menos: proceso vivo, estado cargado (`accounts !== undefined`), y opcionalmente, acceso al filesystem de estado.
- Sin límite de memoria, el memory leak del log crece hasta consumir toda la RAM del host, afectando también a los otros servicios (graphite, grafana, nginx).

3.6.5. Mejoras propuestas

Ordenadas por costo de implementación:

1. **Endpoint de health** (`/health`) en la app + `healthcheck` en `docker-compose.yml`. Costo bajo, permite detectar el estado degradado antes de impactar al usuario.
2. `restart: unless-stopped` en el servicio api. Costo nulo, mitiga el MTTR ante crashes.

3. **Límites de recursos** (`mem.limit`, `cpus`) para contener el blast radius del memory leak.
4. **Keepalive y timeouts en Nginx** para evitar la saturación de sockets y liberar workers ante upstreams lentos.
5. **Node.js cluster** (módulo `cluster` o PM2) para aprovechar múltiples CPUs dentro del mismo contenedor. No resuelve el problema de SPOF ni el de state sharing, pero mejora la capacidad de un nodo único.
6. **Múltiples réplicas + state externalizado (Redis)** para escalado horizontal real. Es la solución definitiva pero requiere refactorizar `state.js` para eliminar el estado en memoria por proceso.

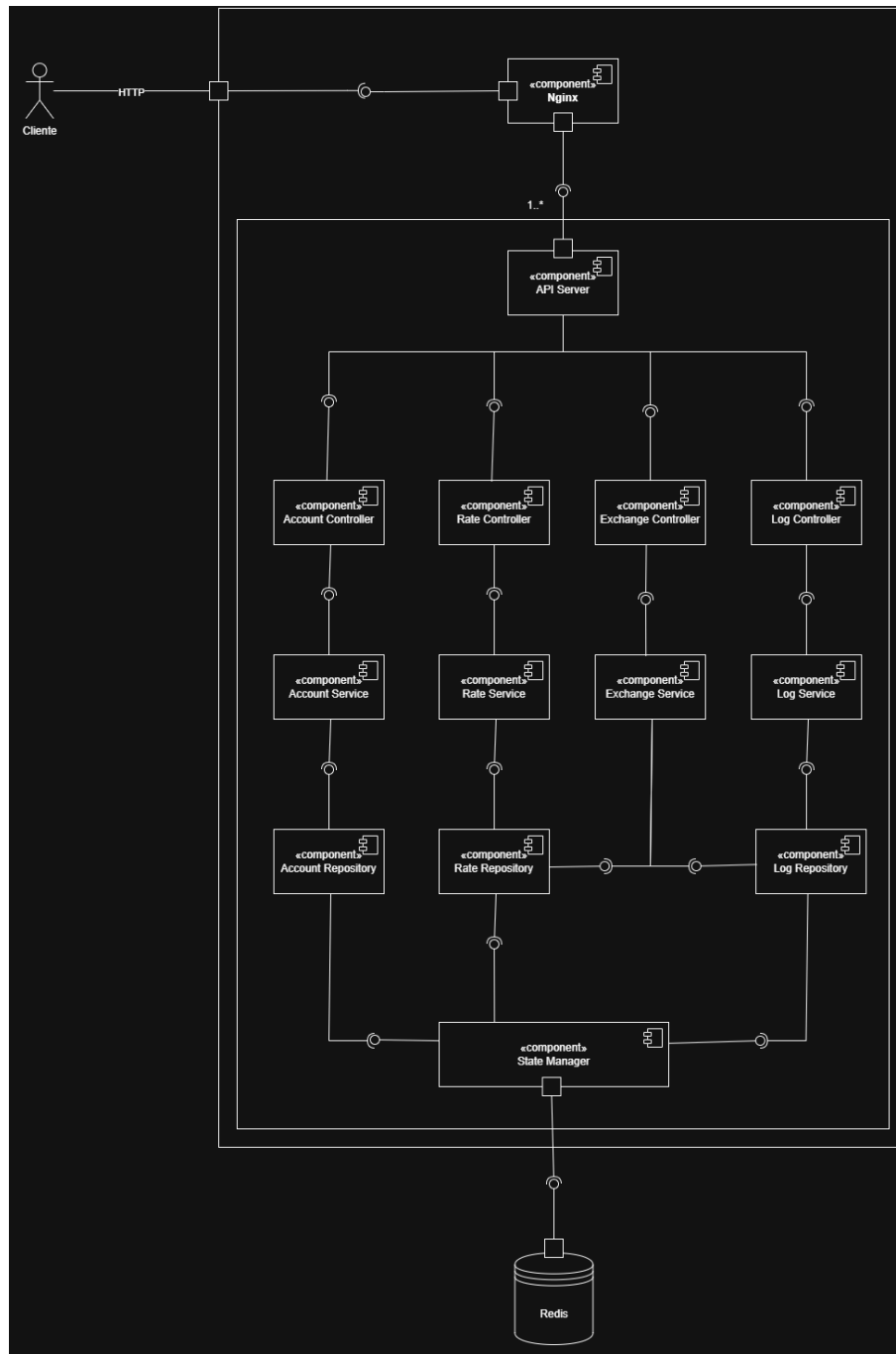


Diagrama 3.4: El único cambio respecto al diagrama anterior es la multiplicidad 1..* entre **Nginx** y la aplicación: ahora pueden existir múltiples instancias, cada una con su propia cadena de Controllers, Services y Repositories, todas compartiendo el mismo estado en Redis.

Las tácticas 1–4 son mejoras quirúrgicas con impacto inmediato sobre Availability y Performance. La táctica 6 es la única que escala Scalability más allá de un solo nodo, pero está bloqueada por la decisión 3.1 (persistencia en JSON local).

3.6.6. Resumen de impacto sobre QA

QA	Impacto	Razón principal
Availability	↓↓↓	SPOF + sin healthcheck + sin restart policy + sin límites de recursos
User-Perceived Performance	↓↓	Event loop único compartido por todos los requests; sin keepalive
Efficiency	↓	Sin keepalive, cada request abre TCP nuevo; GC thrashing invisible
Scalability	↓↓↓	Upstream hardcodeado + estado local impiden réplicas
Visibility	↓	Sin healthcheck no se distingue “running” de “healthy”; modos de falla invisibles
Portability	↓	Nombre de contenedor generado por Compose hardcodeado en Nginx
Testability	↓	Sin endpoint de health, los tests smoke post-deploy no tienen contra qué verificar

3.7. HTTP 500 para errores de negocio

Archivo: app/app.js:88{92

```
if (exchangeResult.ok) {
  res.status(200).json(exchangeResult);
} else {
  res.status(500).json(exchangeResult); // fondos insuficientes → 500
}
```

Una operación rechazada por fondos insuficientes retorna HTTP 500 Internal Server Error, el mismo código que un crash del servidor.

3.7.1. Atributos de calidad afectados

3.7.1.1 Visibility

El panel “Requests State” del dashboard de Grafana agrupa en “Errors” tanto los crashes de infraestructura como los rechazos de negocio normales. Es imposible saber si la tasa de error aumentó porque el servidor está fallando o porque los usuarios se quedan sin fondos. Las alertas sobre ese panel son inherentemente ruidosas e inaccionables.

3.7.1.2 Usabilidad

Las apps mobile que consumen esta API deben tratar el `ok: false` dentro del body JSON para distinguir el rechazo de negocio del error de servidor, ya que el código HTTP no lo informa correctamente. La mayoría de los frameworks HTTP cliente (y las apps de los usuarios) interpretan un 500 como “el servidor rompió”, no como “no tenés fondos”, generando mensajes de error confusos para el usuario final. El código correcto sería HTTP 422 (Unprocessable Entity) para rechazos de negocio.

3.8. Log de transacciones ilimitado en memoria

Archivos: app/exchange.js:108, app/state.js:19, 23

```
log.push(exchangeResult);           // exchange.js:108 | sin límite ni rotación
scheduleSave(log, LOG, 1000);       // state.js:23 | serializa el array completo cada 1s
```

GET /log retorna el array completo sin paginación ni filtros.

3.8.1. Mejora propuesta

Con la incorporación de Redis se eliminó el crecimiento ilimitado del array en memoria. El log de transacciones se almacena en Redis, que permite configurar políticas de expiración y limitar el tamaño de las estructuras de datos, resolviendo el memory leak.

3.8.2. Atributos de calidad afectados

3.8.2.1 Network Performance

El payload de GET /log crecía linealmente con el número de transacciones. En producción con carga sostenida, una respuesta que inicialmente pesa kilobytes puede llegar a megabytes, saturando el ancho de banda del cliente y el tiempo de serialización del servidor. Con Redis es posible implementar paginación y limitar el tamaño de la respuesta, eliminando este problema.

3.8.2.2 Efficiency

JSON.stringify(log) ejecutado cada 1 segundo consumía CPU y tiempo proporcional al tamaño del array. Con millones de entradas, esta operación competía activamente con el procesamiento de requests, degradando la latencia del sistema completo. Con Redis la serialización periódica del array completo desaparece: cada entrada se persiste individualmente en el momento de la operación.

3.8.2.3 Availability

El crecimiento ilimitado del array en memoria podía eventualmente agotar el heap de Node.js y hacer que el OS terminara el proceso. Con Redis el log vive fuera del proceso de Node.js, eliminando el memory leak y el riesgo de OOM asociado.

3.8.2.4 Usabilidad

Sin paginación ni filtros, GET /log era inutilizable para cualquier caso de uso real (auditoría, búsqueda de una transacción específica, consulta de operaciones de un usuario). Redis permite consultas sobre rangos y claves sin cargar todo el log en memoria.

3.9. Impacto de las Decisiones de Diseño sobre los QA

3.9.1. Sistema original

Cuadro 1: Sistema original: impacto sobre disponibilidad y rendimiento

Decisión de diseño	Avail.	Perf. Red	Perf. UX	Efficiency
Evolución: del monolito acoplado a arquitectura en capas	—	—	—	—
Variables en memoria	—	—	↑	—
Persistencia lazy	—	—	↑	—
Persistencia en archivos JSON	↓↓	—	—	↓↓

Tabla 1 (continuación)

Decisión de diseño	Avail.	Perf. Red	Perf. UX	Efficiency
Función <code>transfer()</code> mockeada de forma secuencial	—	—	↓↓	↓
Instancia única sin redundancia ni healthcheck	↓↓↓	—	↓↓	↓
HTTP 500 para errores de negocio	—	—	—	—
Log de transacciones ilimitado en memoria	↓	↓↓	—	↓↓

Cuadro 2: Sistema original: impacto sobre escalabilidad, seguridad, mantenibilidad e integración

Decisión de diseño	Scalab.	Security	Visibility	Testability	Portability	Interop.	Usability
Evolución: del monolito acoplado a arquitectura en capas	—	—	—	↓↓	—	—	—
Variables en memoria	↓↓↓	—	—	—	—	—	—
Persistencia lazy	—	—	—	—	—	—	—
Persistencia en archivos JSON	↓↓	—	—	↓	—	—	—
Función <code>transfer()</code> mockeada de forma secuencial	—	—	↓↓	↓↓	—	↓↓	—
Instancia única sin redundancia ni healthcheck	↓↓↓	—	↓↓	↓	↓↓	—	—
HTTP 500 para errores de negocio	—	—	↓↓	—	—	—	↓↓↓
Log de transacciones ilimitado en memoria	—	—	—	—	—	—	↓

3.9.2. Luego de las mejoras propuestas

Solo se muestran las decisiones para las cuales se propuso una mejora concreta. Las filas sin mejora propuesta (`transfer()` mockeada, HTTP 500) no se incluyen.

Cuadro 3: Luego de mejoras: impacto sobre disponibilidad y rendimiento

Mejora aplicada	Avail.	Perf. Red	Perf. UX	Efficiency
Arquitectura en capas	—	—	↓	—
Variables en memoria → Redis	—	—	↓	—
Persistencia lazy → Redis	—	—	↓	↓
Persistencia JSON → Redis	↑↑	—	—	↑↑
Instancia única → healthcheck, restart, límites, Nginx	↑↑	—	↑	↑
Log ilimitado → Redis	↑↑	↑↑	—	↑↑

Cuadro 4: Luego de mejoras: impacto sobre escalabilidad, seguridad, mantenibilidad e integración

Mejora aplicada	Scalab.	Security	Visibility	Testability	Portability	Interop.	Usability
Arquitectura en capas	—	—	—	↑↑	—	—	—
Variables en memoria → Redis	↑↑↑	—	—	—	—	—	—

Tabla 4 (continuación)

Mejora aplicada	Scalab.	Security	Visibility	Testability	Portability	Interop.	Usability
Persistencia lazy → Redis	—	—	—	—	—	—	—
Persistencia JSON → Redis	↑↑↑	—	—	↑	—	—	—
Instancia única → healthcheck, restart, límites, Nginx	↑↑	—	↑↑	↑	↑	—	—
Log ilimitado → Redis	—	—	—	—	—	—	↑↑

4. Pruebas

En esta sección se describen los escenarios de prueba, cómo ejecutarlos y cómo interpretar los resultados. El objetivo es validar que el sistema cumple con los requisitos de rendimiento y escalabilidad definidos en la sección 2.

4.0.1. Correr los escenarios

Todos los comandos se ejecutan desde el directorio `perf/`.

4.0.1.1 Escenarios disponibles

Archivo	Endpoint principal	Uso
<code>rates.yaml</code>	GET /rates	Solo lectura, mínima lógica
<code>exchange.yaml</code>	POST /exchange	Operación crítica con lógica de negocio
<code>mixed.yaml</code>	Mix 60/30/5/5	Perfil de tráfico realista

4.0.1.2 Workloads disponibles

Workload	Duración aprox.	Carga	Pregunta que responde
<code>baseline</code>	3 min	1 req/s constante	¿Funciona sin errores con mínima carga?
<code>load</code>	7 min	Ramp 1→20, luego 20 req/s	¿Cómo se comporta bajo carga normal?
<code>stress</code>	10 min	Escalado 10→25→50→100 rps	¿En qué punto aparecen errores o degradación?

4.1. Objetivo

Obtener un panorama cuantitativo y reproducible del comportamiento del servicio bajo distintos regímenes de carga. Las mediciones deben permitir:

1. Establecer un **baseline** del estado actual del sistema antes de aplicar cualquier mejora.
2. Identificar los **puntos de ruptura** (saturación, degradación, crash).
3. Prover la **línea de comparación** contra la cual medir el impacto de las tácticas que se implementen en etapas posteriores.

4.2. Workload Models (Modelos de Carga)

Basado en Artillery Workload Models y la distinción load vs stress testing de los artículos provistos, se proponen 3 modelos diferenciados. Cada uno responde a una pregunta distinta.

4.2.1. Baseline (Smoke / Low-Constant)

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima, sin estrés?

- **Duración:** 3 minutos.
- **Carga:** 1 req/s constante.
- **Propósito:** Medir latencia base, verificar que no hay errores en condiciones ideales. Usado como referencia para comparar con cargas mayores.

4.2.2. Load Test (Carga Normal Sostenida)

Pregunta: ¿Cómo se comporta el sistema bajo la carga esperada en producción?

- **Duración:** 10 minutos.
- **Carga:** Ramp 1→20 req/s en 2 min, constante 20 req/s durante 8 min.
- **Propósito:** Observar comportamiento estable bajo carga esperada. Los problemas deben manifestarse como latencia elevada o tasa de error creciente.

4.2.3. Stress Test (Buscar el Punto de Quiebre)

Pregunta: ¿Cuál es la capacidad máxima del sistema antes de degradarse?

- **Duración:** 15 minutos.
- **Carga:** Ramp 1→200 req/s escalonado (10, 25, 50, 100, 200 req/s, 3 min en cada escalón).
- **Propósito:** Identificar el knee point de la curva de latencia y el RPS al que aparecen los primeros errores. Exhibe la race condition de `exchange()`.

4.3. Escenarios de Artillery a Implementar

Cada uno de los 3 workload models se ejecutará sobre cada uno de los siguientes escenarios para aislar el comportamiento por tipo de operación.

4.3.1. perf/rates.yaml (dado por la cátedra)

- **Target:** GET /rates
- **Característica:** Solo lectura, no ejercita persistencia ni transfers. Baseline del costo de un request sin lógica.

4.3.2. perf/exchange.yaml (nuevo)

- **Target:** POST /exchange con bodies variados (distintos pares de monedas y montos).
- **Característica:** Operación crítica. Expone race condition, latencia de 400–800 ms del mock `transfer()`, escritura eventual del log.
- **Payload:** rotación entre pares ARS/USD, USD/EUR, BRL/ARS con `baseAmount` aleatorio en un rango realista.

4.3.3. perf/mixed.yaml (nuevo)

- **Target:** Mix realista de endpoints con pesos:
 - 60 % GET /rates
 - 30 % POST /exchange
 - 5 % GET /accounts
 - 5 % GET /log
- **Característica:** Representa el perfil de tráfico real. Expone la falta de bulkhead: GET /log bloquea el event loop y degrada a los POST /exchange.

4.4. Plan de Ejecución

4.4.1. Fase A – Baseline

- **Objetivo:** Medir comportamiento del sistema sin cambios operativos ni de arquitectura.
- **Ejecución:**
 1. Ejecutar rates en los tres workload models (baseline, load, stress). Guardar resultados y tomar screenshot tras cada ejecución.
 2. Ejecutar exchange (baseline → load → stress) de la misma manera.
 3. Ejecutar mixed (baseline → load → stress). Pausar y capturar evidencias.
 4. Recolectar logs y guardar si se detectan errores.

4.4.2. Fase B – Redis + arquitectura de capas + healthcheck + keepalive

- **Objetivo:** Validar que la migración a Redis y mejoras operativas corrigen errores de persistencia y mejoran disponibilidad.
- **Ejecución:**
 1. Repetir los mismos runs que en Fase A: rates, exchange, mixed (baseline/load/stress), en el mismo orden.
 2. Antes de cada run largo, comprobar logs para detectar errores inmediatos.
 3. Guardar resultados y capturas.
- **Validaciones:**
 - Comparar métricas vs Fase A y registrar mejora o regresión.

4.4.3. Fase C – Escalado horizontal (3 instancias)

- **Objetivo:** Evaluar el efecto del escalado horizontal (3 réplicas) sobre latencias y errores en cargas mixtas y de stress.
- **Ejecución:**
 1. Ejecutar exchange (load → stress). Guardar resultados y capturas.
 2. Ejecutar mixed (load → stress). Guardar resultados y capturas.
- **Validaciones y artefactos:**
 - Confirmar reducción en errores 5xx/ETIMEDOUT respecto a Fase A/B.
 - Registrar p50/p95/p99 y throughput; generar tablas comparativas (Fase A vs B vs C).

4.5. Análisis de resultados

Esta sección resume los resultados obtenidos en las ejecuciones de ‘baseline’, ‘load’ y ‘stress’ para los escenarios ‘rates’, ‘exchange’ y ‘mixed’ en cada una de las fases.

4.5.1. Escenario ‘rates’ (GET /rates)

4.5.1.1 Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima?

Resultado: requests=390, rps≈4, p50=2 ms, p95=80.6 ms, p99=113.3 ms, fallos=0.

Respuesta: Bajo carga mínima el endpoint responde muy rápido (mediana 2 ms) y sin errores.

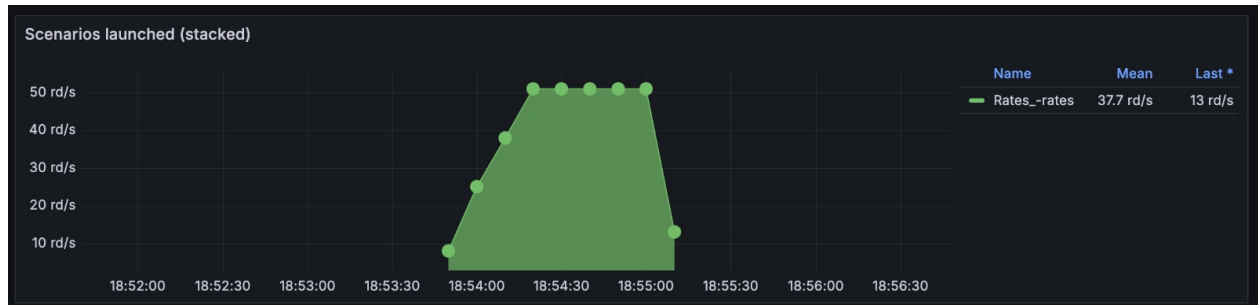


Imagen 4.1: Gráfica de escenarios - Baseline - rates

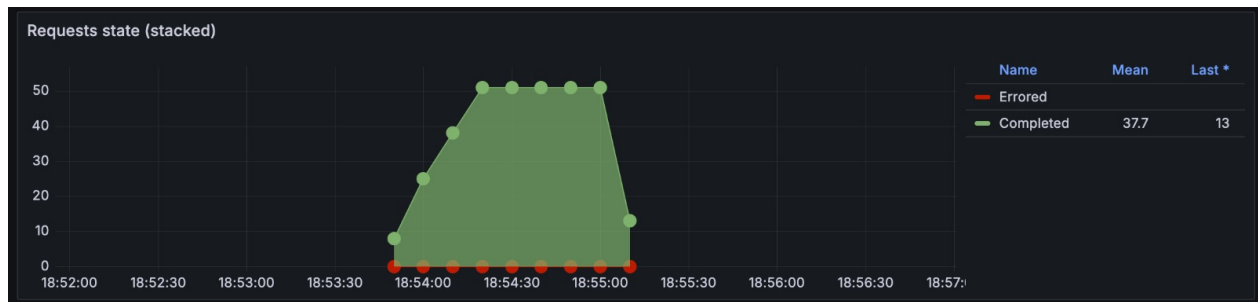


Imagen 4.2: Gráfica de estado de solicitudes - Baseline - rates

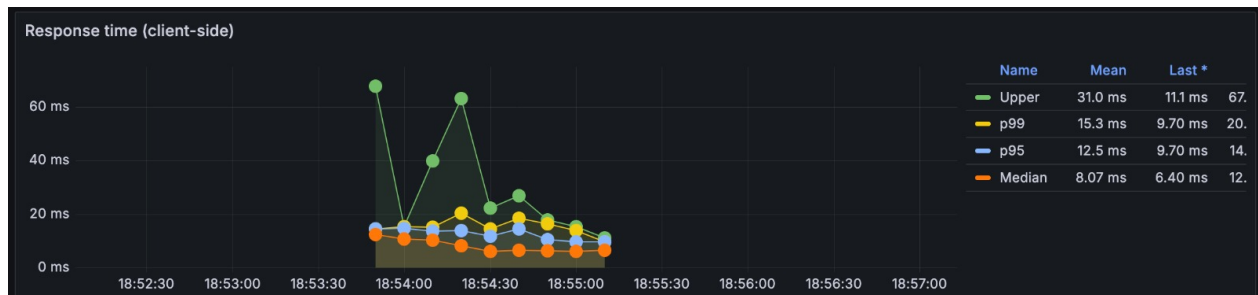


Imagen 4.3: Gráfica de respuestas - Baseline - rates

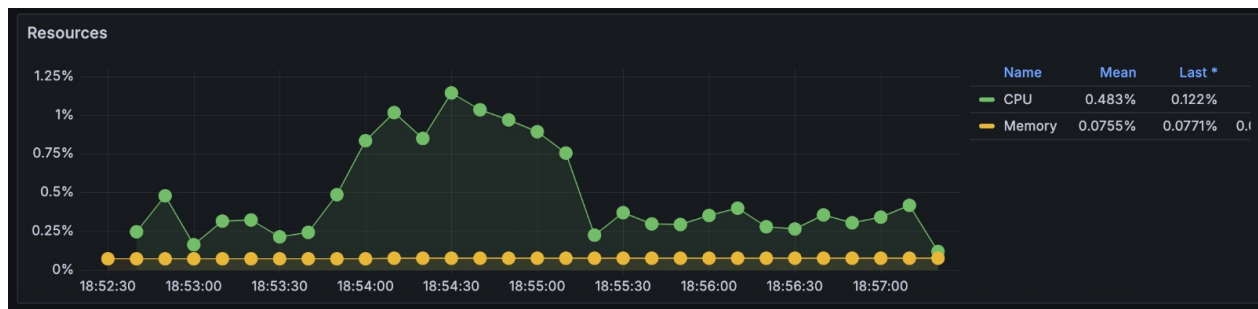


Imagen 4.4: Gráfica de recursos - Baseline - rates

4.5.1.2 Load

Pregunta: ¿Cómo se comporta bajo carga esperada (ramp → 20 rps)?

Resultado: requests=7260, rps≈17, p50=2 ms, p95=25.8 ms, p99=67.4 ms, fallos=0.

Respuesta: A 17–20 rps sostenidos el servicio mantiene latencias muy bajas y sin errores. Comportamiento estable.

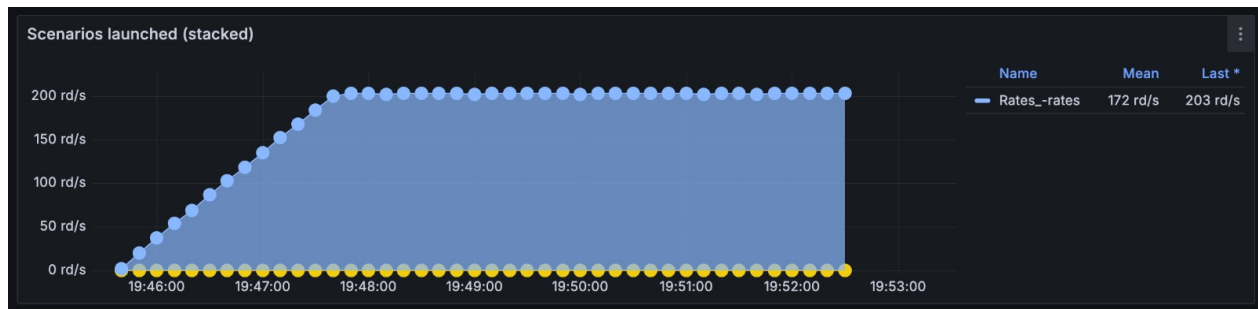


Imagen 4.5: Gráfica de escenarios - Load - rates

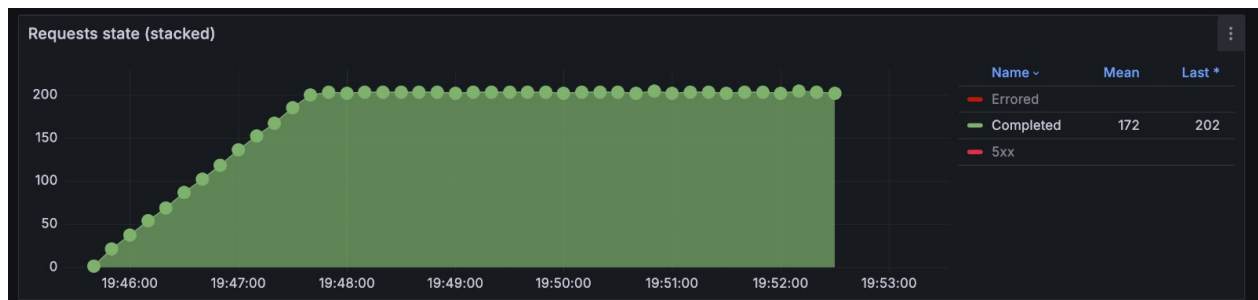


Imagen 4.6: Gráfica de estado de solicitudes - Load - rates

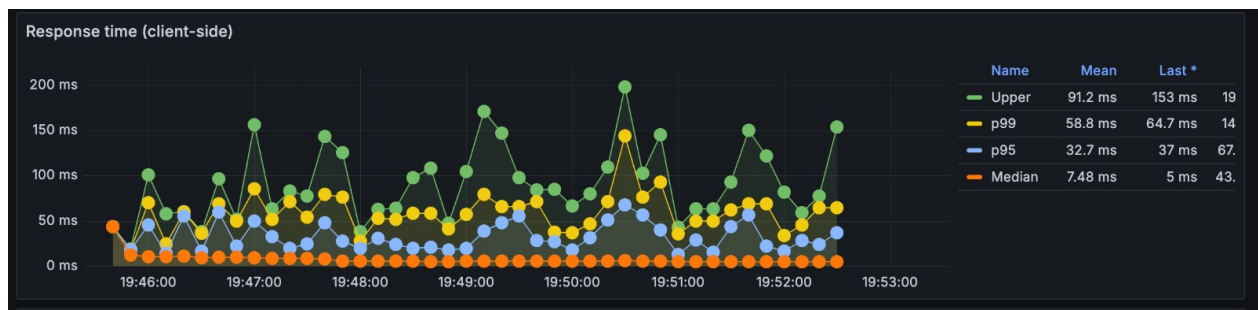


Imagen 4.7: Gráfica de respuestas - Load - rates

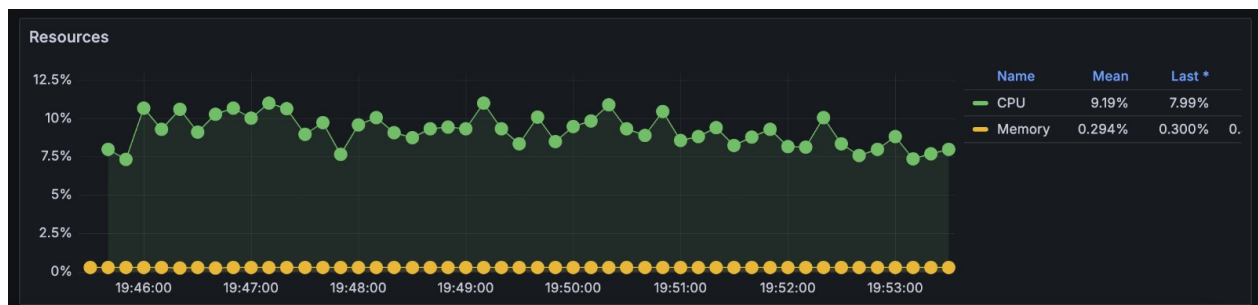


Imagen 4.8: Gráfica de recursos - Load - rates

4.5.1.3 Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse?

Resultado: requests=45900, rps≈77, p50=3 ms, p95=122.7 ms, p99=179.5 ms, fallos=0.

Respuesta: 'rates' escala bien hasta los niveles de stress probados (promedio ~77 rps en la ejecución). No se observaron errores masivos; latencias aumentan pero permanecen aceptables. Punto de ruptura no alcanzado en estas pruebas.

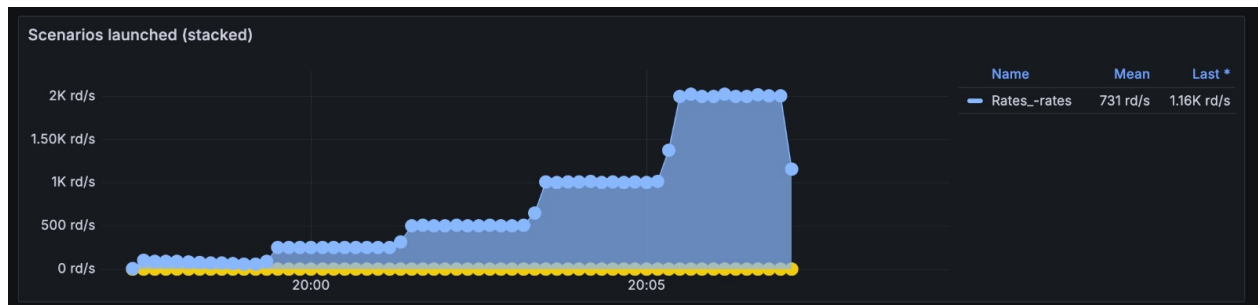


Imagen 4.9: Gráfica de escenarios - Stress - rates

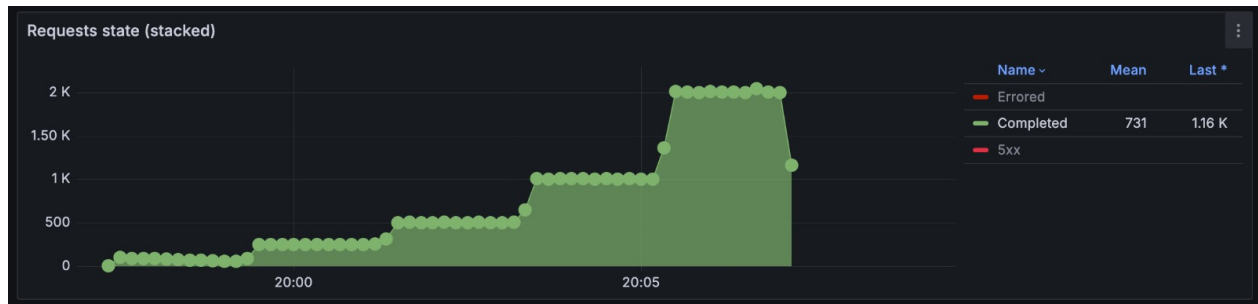


Imagen 4.10: Gráfica de estado de solicitudes - Stress - rates

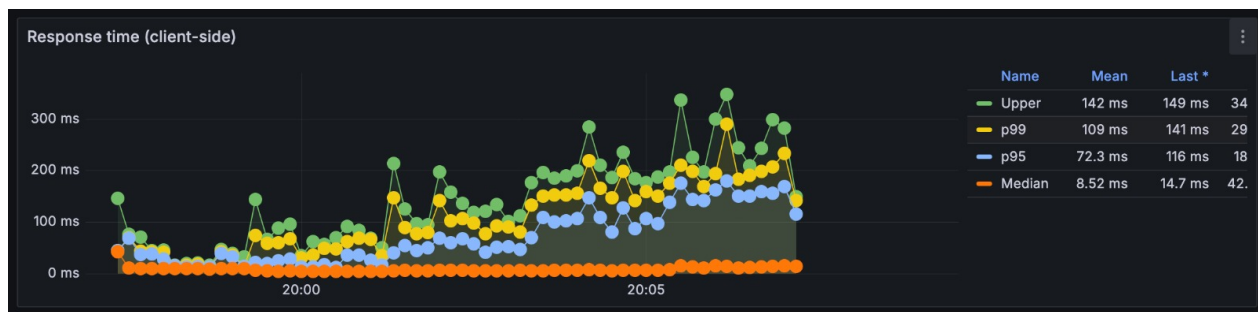


Imagen 4.11: Gráfica de respuestas - Stress - rates

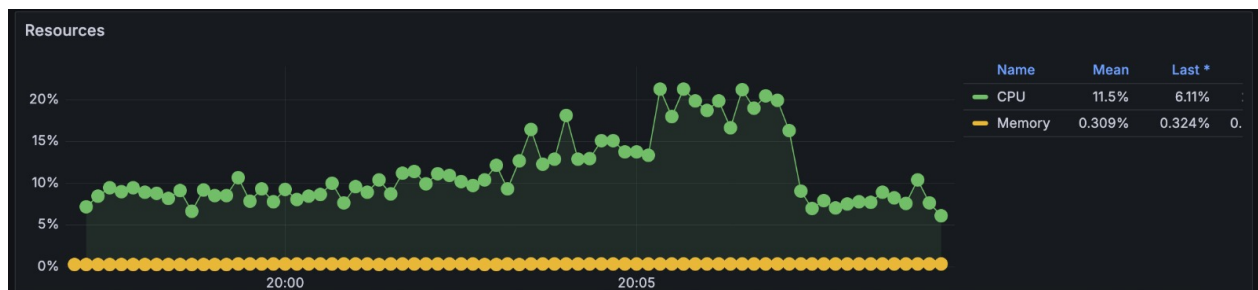


Imagen 4.12: Gráfica de recursos - Stress - rates

4.5.1.4 Conclusión

Endpoint robusto y de baja latencia; no es el cuello de botella.

4.5.2. Escenario ‘exchange’ (POST /exchange)

4.5.2.1 Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima?

Resultado: requests=180, rps=1, p50≈596 ms, p95≈727.9 ms, p99≈772.9 ms, fallos=0.

Respuesta: Incluso en baseline el endpoint tiene latencia alta (≈600 ms mediana) atribuible a la lógica de negocio simulada.

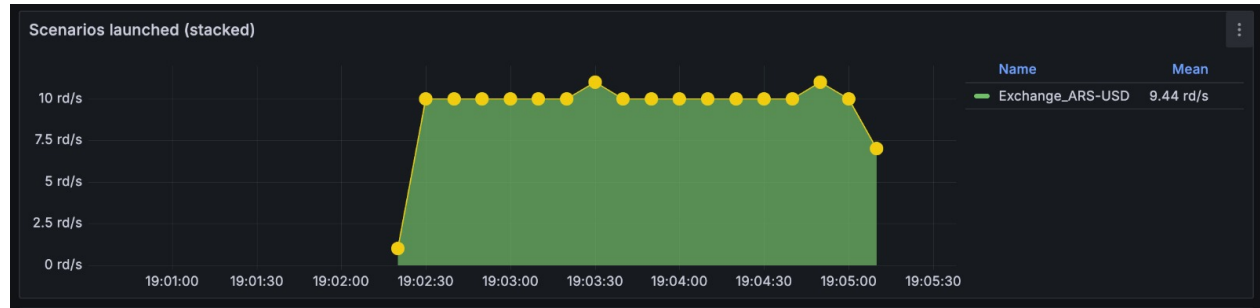


Imagen 4.13: Gráfica de escenarios - Baseline - exchange

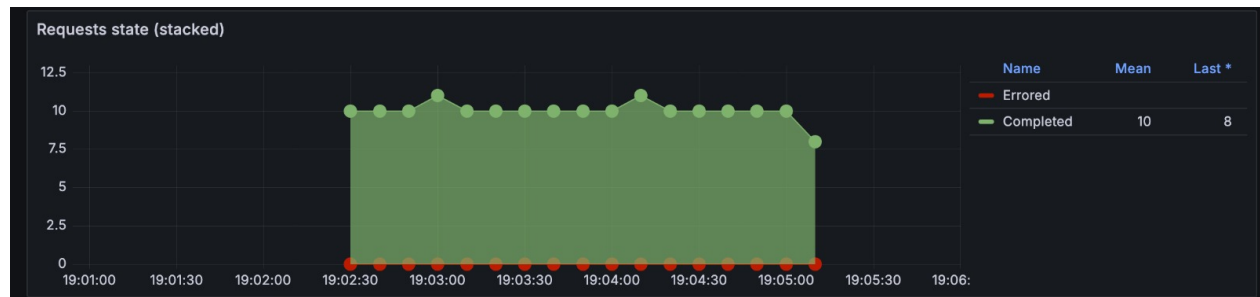


Imagen 4.14: Gráfica de estado de solicitudes - Baseline - exchange

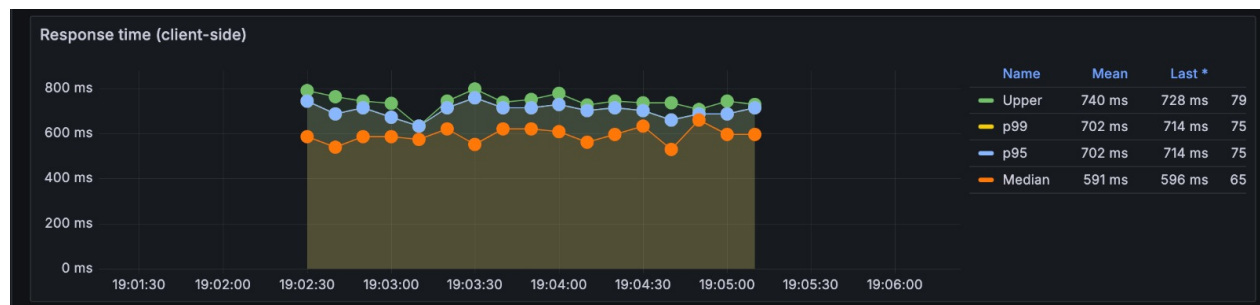


Imagen 4.15: Gráfica de respuestas - Baseline - exchange

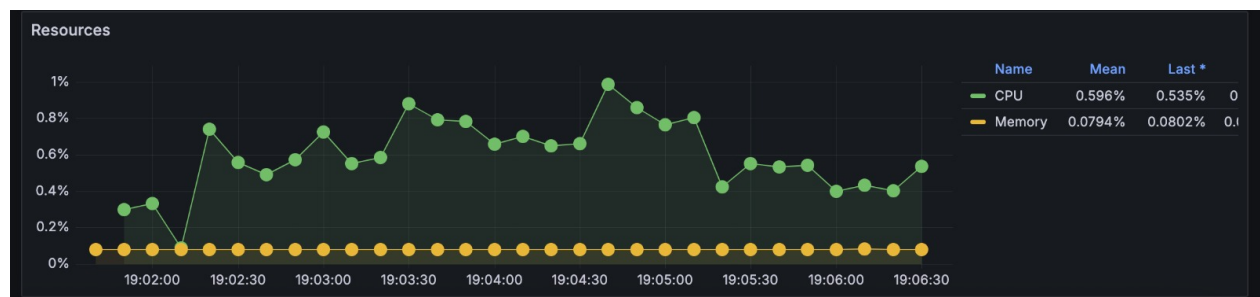


Imagen 4.16: Gráfica de recursos - Baseline - exchange

4.5.2.2 Load

Pregunta: ¿Cómo se comporta bajo carga esperada (20 rps)?

Resultado: requests=7260, rps≈17, p50≈620.3 ms, p95≈757.6 ms, p99≈820.7 ms, fallos=0.

Respuesta: A 17-20 rps la mediana/percentiles se mantienen en el rango de cientos de ms. La cola y latencia se elevan notablemente comparado con 'rates'.

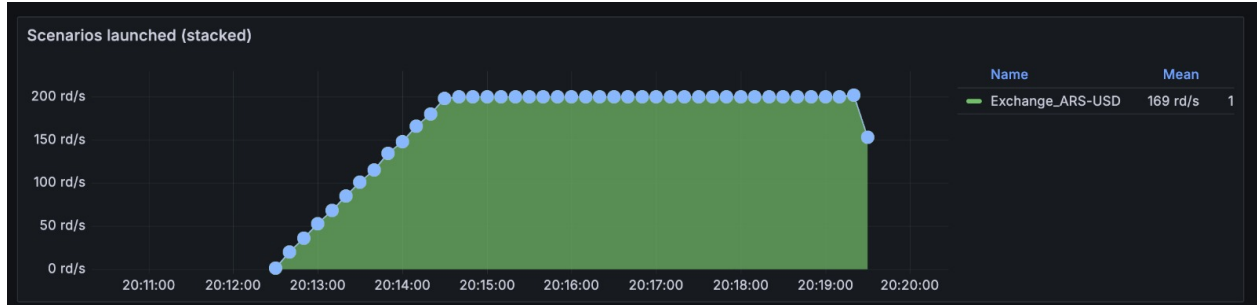


Imagen 4.17: Gráfica de escenarios - Load - exchange

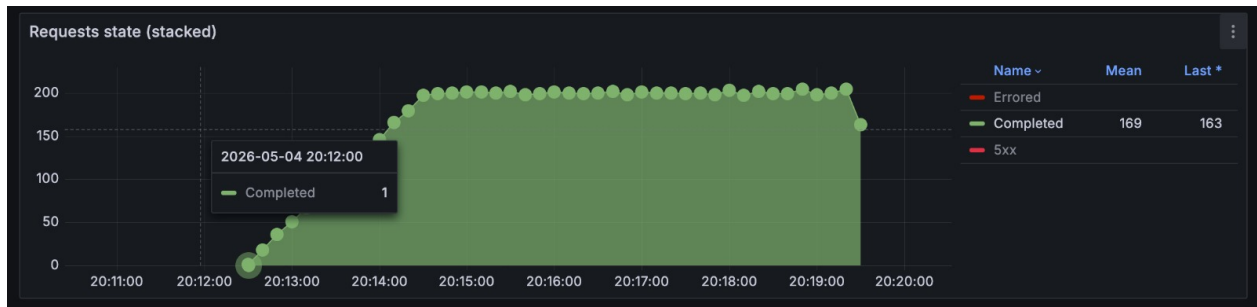


Imagen 4.18: Gráfica de estado de solicitudes - Load - exchange

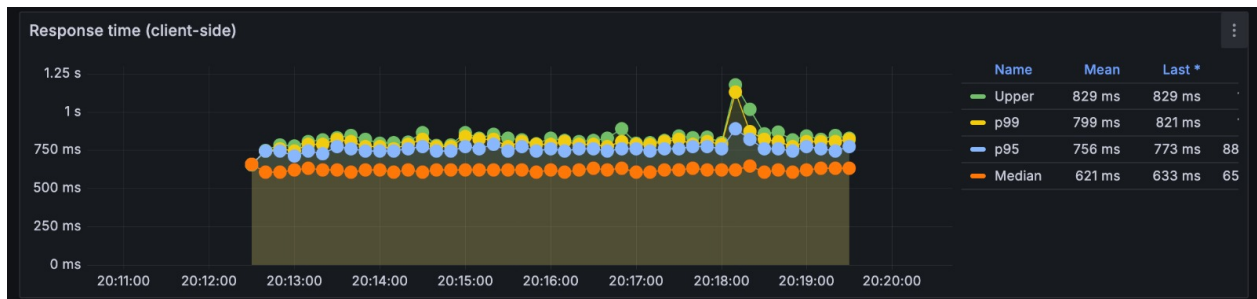


Imagen 4.19: Gráfica de respuestas - Load - exchange

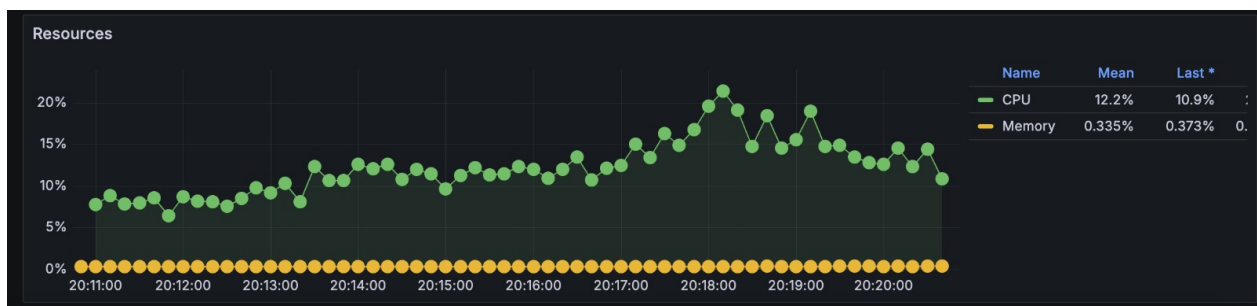


Imagen 4.20: Gráfica de recursos - Load - exchange

4.5.2.3 Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse?

Resultado: requests=46200, rps≈77; vusers.failed=11601; errores: 'ERR_SOCKET_TIMEOUT'≈10467, 'ECONNRESET'≈1134, http.5xx≈52; p95≈3905.8 ms, p99≈7407.5 ms.

Respuesta: Bajo stress el endpoint se degrada gravemente: aparecen timeouts y resets con altas latencias (segundos). El servicio no puede sostener cargas elevadas.

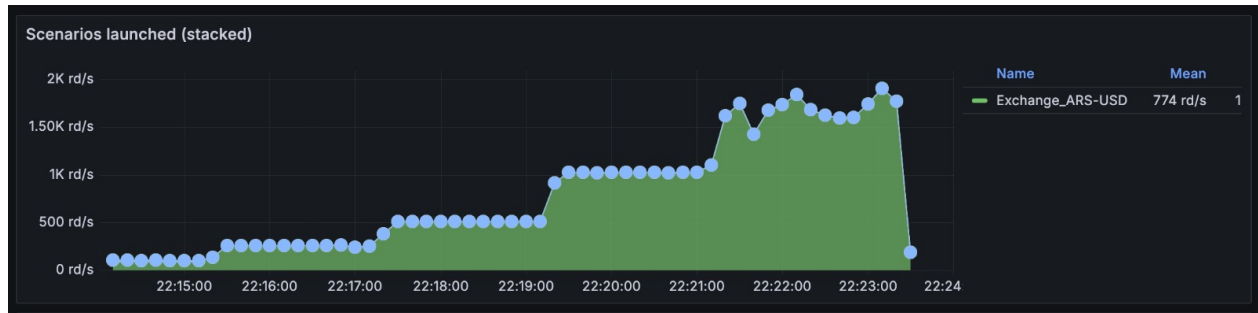


Imagen 4.21: Gráfica de escenarios - Stress - exchange

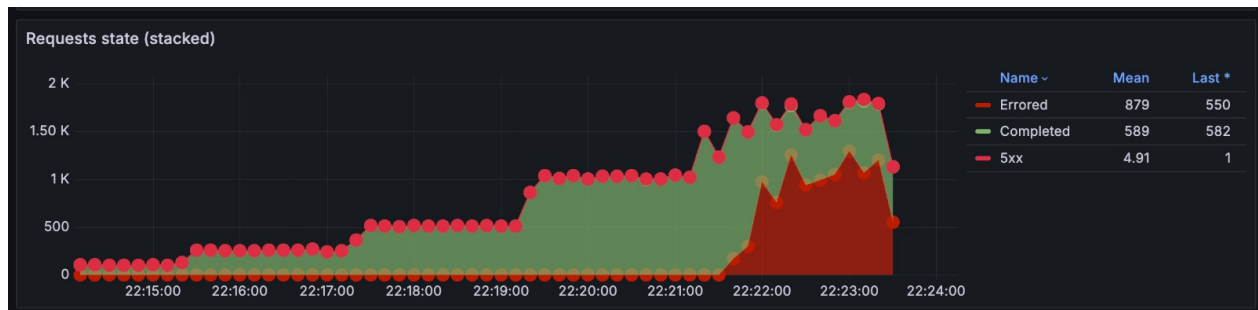


Imagen 4.22: Gráfica de estado de solicitudes - Stress - exchange

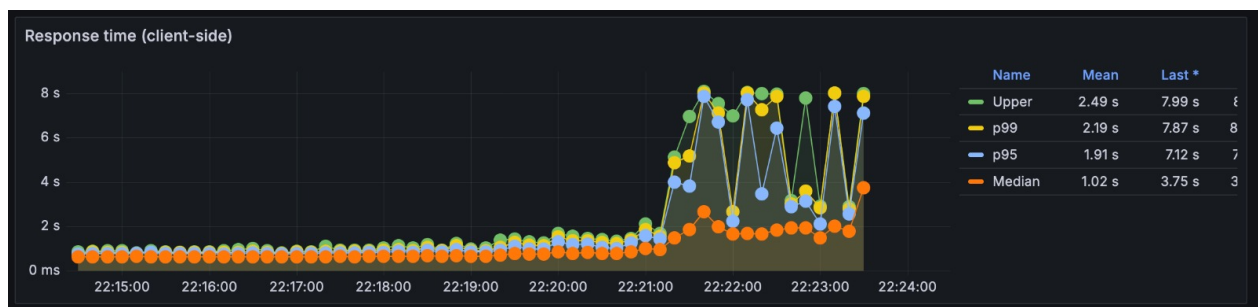


Imagen 4.23: Gráfica de respuestas - Stress - exchange

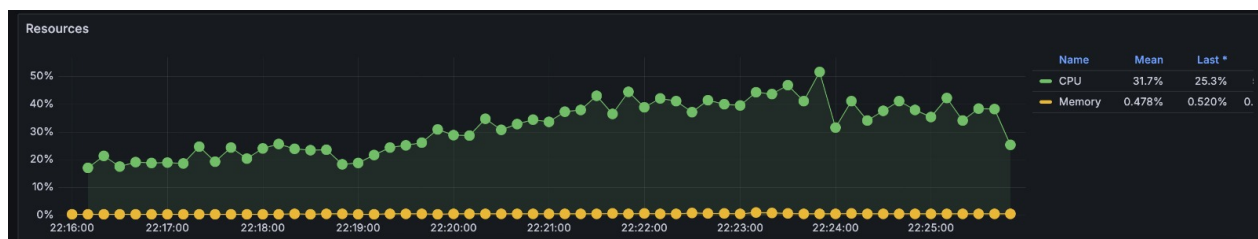


Imagen 4.24: Gráfica de recursos - Stress - exchange

4.5.2.4 Conclusión

Es el cuello de botella. Su latencia base (~ 600 ms) combinada con la naturaleza síncrona produce colas y timeouts bajo carga. Requiere optimizaciones.

4.5.3. Escenario 'mixed' (mix 60 % rates / 30 % exchange / 5 % accounts / 5 % log)

4.5.3.1 Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima en un perfil realista?

Resultado: requests=720, rps $\approx$ 4, p50=3 ms, p95 $\approx$ 685.5 ms, p99 $\approx$ 742.6 ms, fallos=0.

Respuesta: La mediana es baja (por predominio de 'rates') pero los percentiles altos reflejan el impacto de los exchanges.

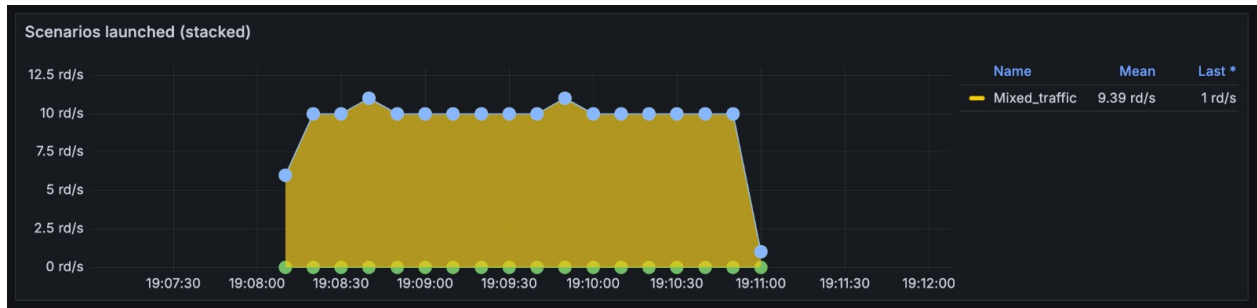


Imagen 4.25: Gráfica de escenarios - Baseline - mixed

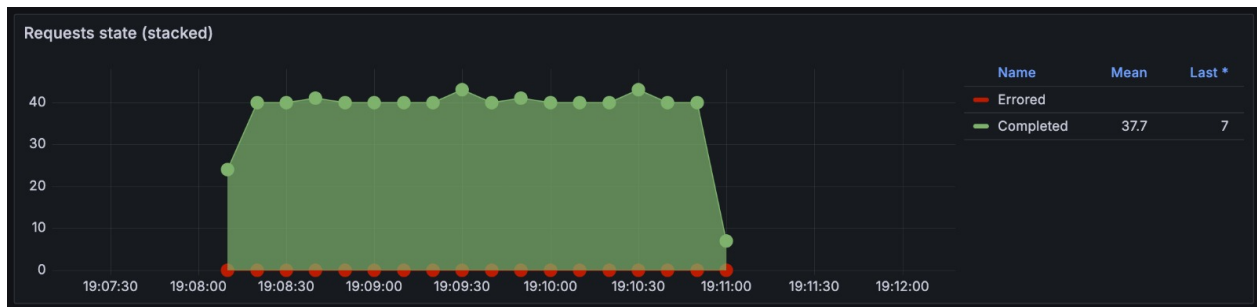


Imagen 4.26: Gráfica de estado de solicitudes - Baseline - mixed

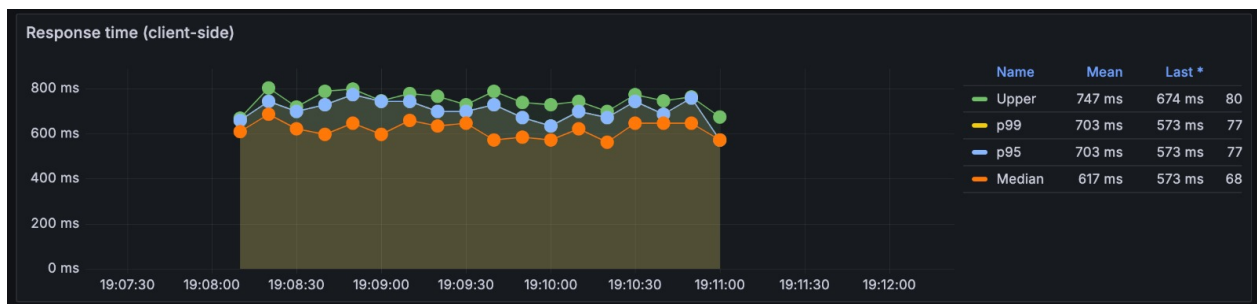


Imagen 4.27: Gráfica de respuestas - Baseline - mixed

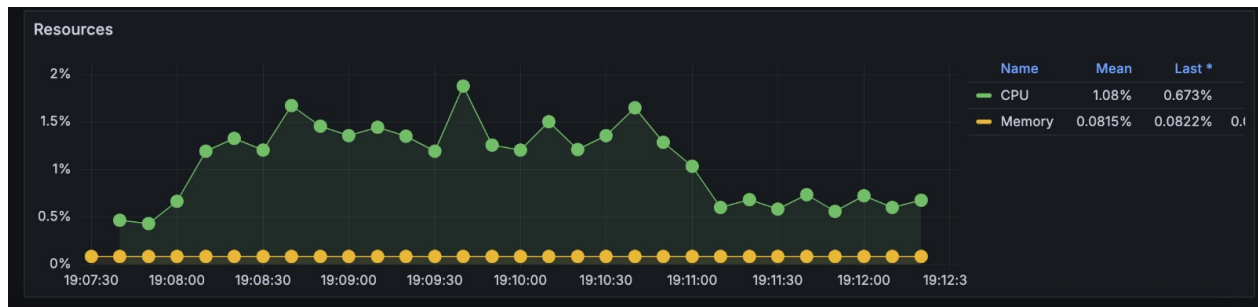


Imagen 4.28: Gráfica de recursos - Baseline - mixed

4.5.3.2 Load

Pregunta: ¿Cómo se comporta bajo carga esperada (ramp → 20 rps total)?

Resultado: requests=29040, rps≈69 (total mixto), p50≈49.9 ms, p95≈837.3 ms, p99≈1249.1 ms, fallos=0.

Respuesta: Con mezcla realista la latencia se eleva (p95 cercano a 0.8–1.2 s). No hay fallos masivos, pero la degradación de experiencia es evidente.

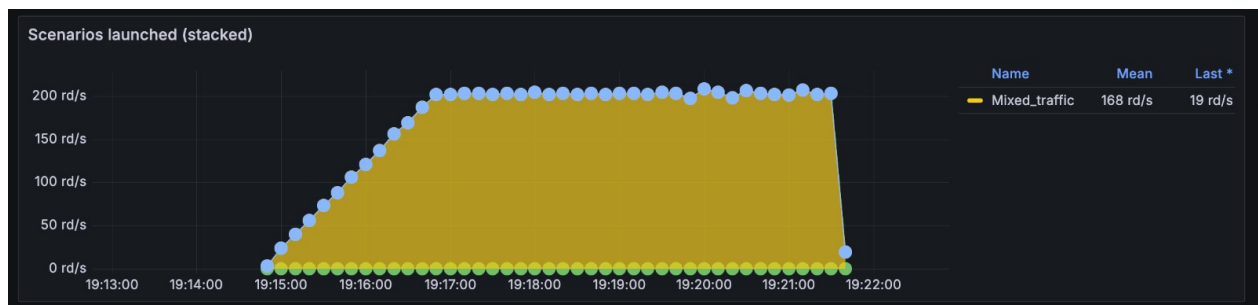


Imagen 4.29: Gráfica de escenarios - Load - mixed



Imagen 4.30: Gráfica de estado de solicitudes - Load - mixed

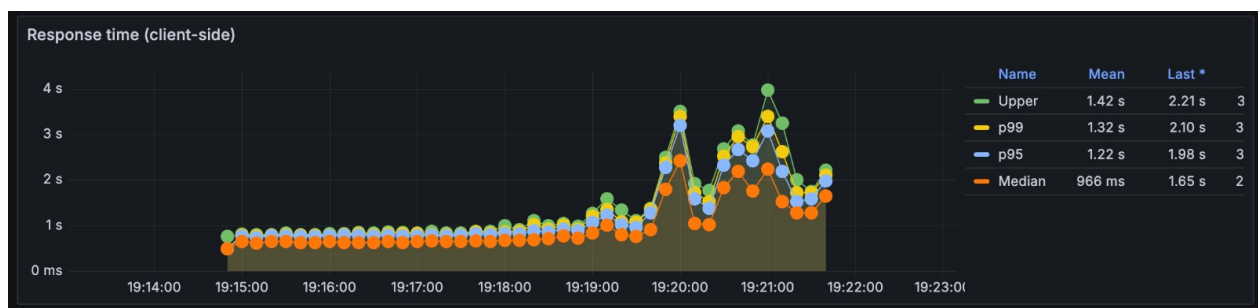


Imagen 4.31: Gráfica de respuestas - Load - mixed

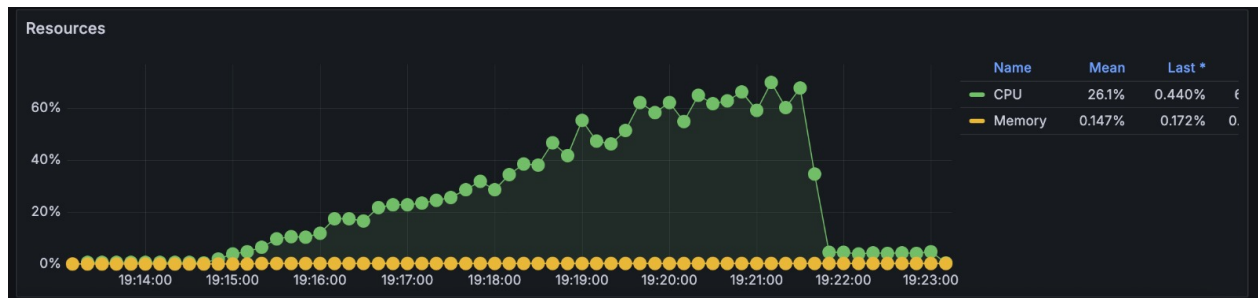


Imagen 4.32: Gráfica de recursos - Load - mixed

4.5.3.3 Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse en tráfico mixto?

Resultado: requests=76751, rps≈77; http.codes.200≈37393, vusers.failed≈39321; errores: 'ERR_SOCKET_TIMEOUT'≈3, 'ECONNRESET'≈401, http.5xx≈97; p95≈7407.5 ms, p99≈8186.6 ms.

Respuesta: En stress combinado el sistema falla de forma masiva con tiempos de respuesta en segundos y tasas de fallo muy altas.

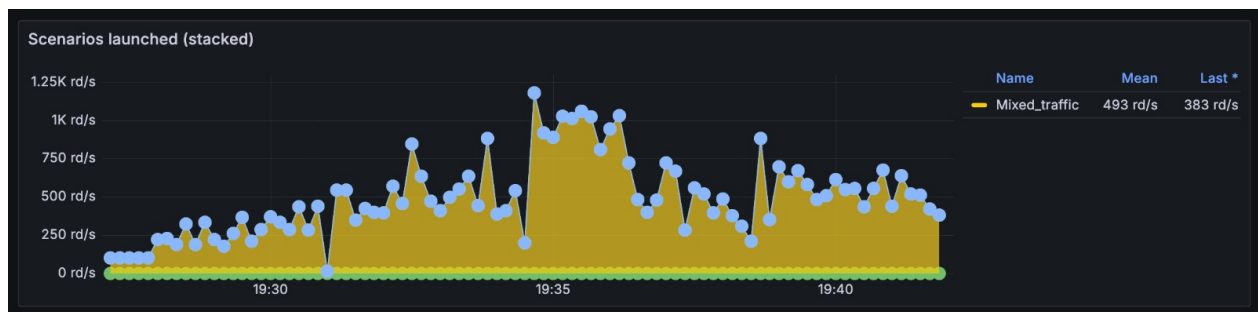


Imagen 4.33: Gráfica de escenarios - Stress - mixed

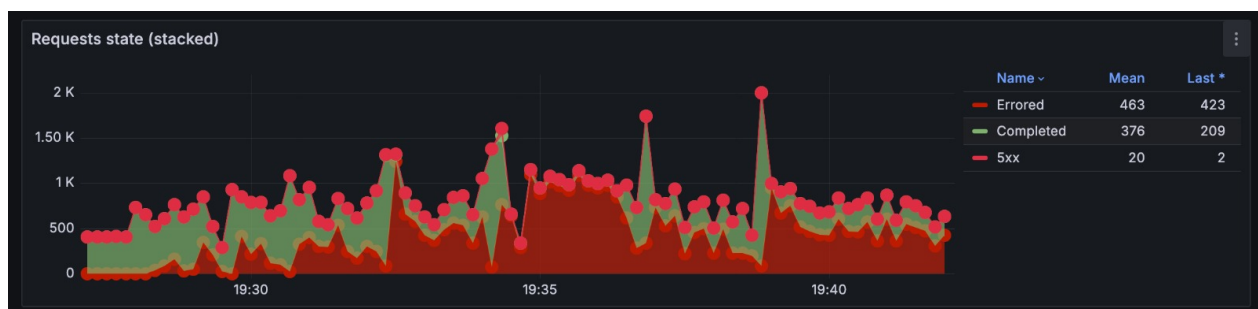


Imagen 4.34: Gráfica de estado de solicitudes - Stress - mixed

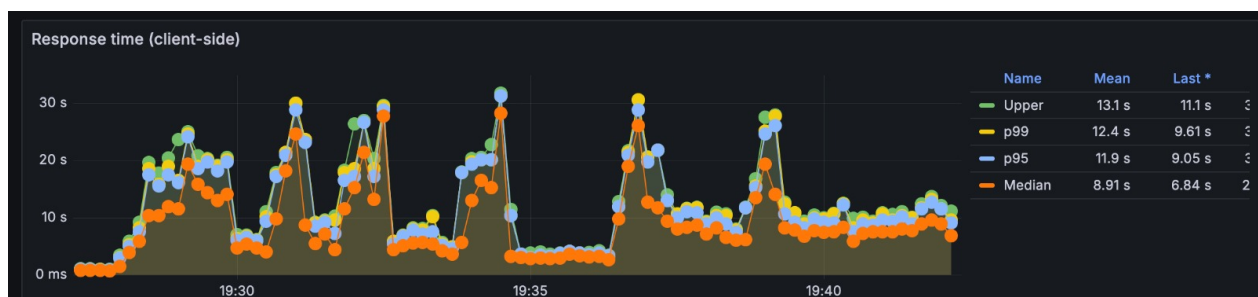


Imagen 4.35: Gráfica de respuestas - Stress - mixed

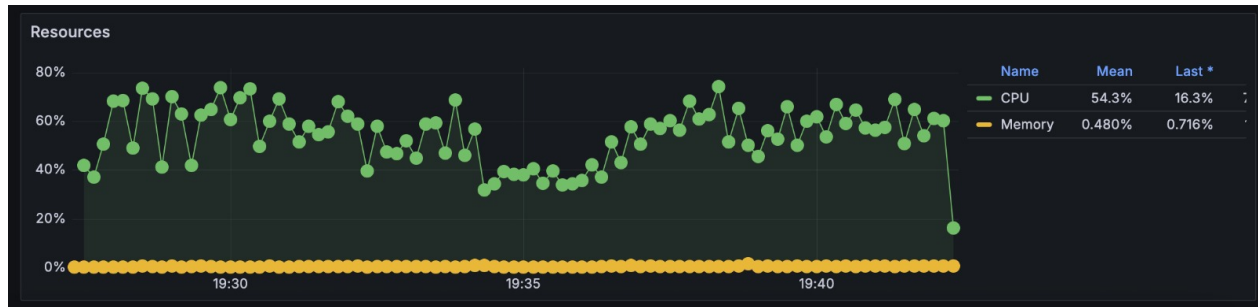


Imagen 4.36: Gráfica de recursos - Stress - mixed

4.5.3.4 Conclusión

El comportamiento es dominado por el 'exchange' bajo carga. La presencia de operaciones lentas contamina la experiencia de los endpoints rápidos.

4.6. Puntos De Ruptura Identificados

4.6.1. '/rates' (Read-Only)

- **Baseline:** ✓ 1 req/s — Sin problemas
- **Load:** ✓ 20 req/s — Sin problemas (p99=12ms)
- **Stress:** ★ 50 req/s → OK, pero 100 req/s → **FALLA** (punto de ruptura ~80 req/s)

Capacidad sostenida: ~20-50 req/s. En stress (100 req/s sostenido), error rate sube a 50 %.

4.6.2. '/exchange' (Business Logic)

- **Baseline:** ✓ 1 req/s — OK (6.67 % error por lógica, no por saturación)
- **Load:** ⊗ 20 req/s → **98 % error rate** (punto de ruptura ~5-10 req/s)
- **Stress:** ⊗ 10 req/s → Ya casi saturado

Capacidad sostenida: ~1-2 req/s máximo. El endpoint es el cuello de botella del sistema.

4.6.3. '/mixed'

- **Baseline:** ✓ 1 req/s — OK
- **Load:** ⊗ ~15 req/s (equivalente a 4.5 exchange/s + 9 reads/s) → Empieza a fallar
- **Stress:** ⊗ 100 req/s → 99 % error

Capacidad sostenida: ~10 req/s máximo (mezcla con 30 % exchange).

4.7. Análisis Profundo: ¿Por Qué Falla?

4.7.1. Latencia Base de '/exchange'

La operación exchange incluye:

1. Validación: ~1ms
2. Primer transfer (mock): 200-400ms ← **CUELLO DE BOTELLA**
3. Segundo transfer (mock): 200-400ms ← **CUELLO DE BOTELLA**

4. Actualización de balances: $\sim 1\text{ms}$

5. Log push: $\sim 1\text{ms}$

Total: 400-800ms por exchange.

Con **20 exchanges/s** (si toda carga fuera exchange):

- Tiempo total de computación: $20 \times 600\text{ms} = 12,000\text{ms} = 12$ segundos de backlog
- Pero Node.js single-threaded solo puede procesar 1 cosa a la vez
- Si llega un exchange en el ms 1, y tarda 600ms, el siguiente tiene que esperar
- Con 20/s, la cola crece indefinidamente

4.7.2. Single-Threaded Event Loop

```
t=0ms:   Req1 inicia -----> t=600ms Req1 completa
         Req2 entra en cola (espera)
         Req3 entra en cola
         ...
         Req20 entra en cola
t=600ms: Req2 inicia -----> t=1200ms Req2 completa
         Pero ahora la cola tiene 31 requests esperando
         Algunos ya esperan > 10s, Artillery los mata por timeout
```

4.7.3. Race Condition

Bajo estrés, dos requests de exchange pueden ejecutarse casi en paralelo en la lógica de actualización de balances:

```
// Thread 1: Exchange A           // Thread 2: Exchange B
baseAccount.balance += baseAmount   (at ~same time)
counterAccount.balance -= counterAmount
```

```
// Si ambos leen el balance al mismo tiempo, pueden escribir
// valores inconsistentes al estado compartido
```

Aunque JavaScript no tiene multithreading verdadero, el evento de `scheduleSave()` cada 1s podría causar que el estado guardado sea inconsistente.

4.8. Fase B (Redis + arquitectura de capas + healthcheck + keepalive)

4.8.1. Resumen rápido

- **exchange (re-run):** 7260 requests, todas 200; mediana ≈ 659 ms, mean ≈ 1171 ms, p95 ≈ 4231 ms, p99 ≈ 7710 ms.
- **rates (re-run):** 7260 requests, todas 200; mediana ≈ 2 ms, mean ≈ 1.8 ms, p95 ≈ 4 ms, p99 ≈ 6 ms.
- **mixed (re-run):** 16999 requests; `http.codes.200=10948`, `http.codes.504=848`, `errors.ETIMEDOUT=5203`; median ≈ 3012 ms, mean ≈ 3859 ms, p95 ≈ 9801 ms.

4.8.2. Análisis: ¿Por qué errores en mixed y por qué exchange es cuello de botella?

- **exchange** tiene latencia base alta por la lógica de negocio simulada (`transfer()`), lo que produce cola bajo RPS combinados.
- Node.js puede verse afectado por operaciones largas o bloqueantes en el event loop; esto aumenta latencias y provoca timeouts en el gateway (504) y en clientes (ETIMEDOUT).
- En **mixed** la mezcla de endpoints rápidos y lentos hace que los endpoints rápidos también degraden su rendimiento por saturación de recursos y/o saturación de Redis si hay muchas escrituras.

4.8.3. Respuestas por escenario (Fase B)

4.8.3.1 Escenario rates

Pregunta: ¿Cómo se comporta bajo carga esperada?

Resultado: requests=7260, rps≈20, p50=2 ms, p95=4 ms, p99=6 ms, fallos=0.

Respuesta: rates mantiene latencias muy bajas y sin errores tras la corrección (Redis).

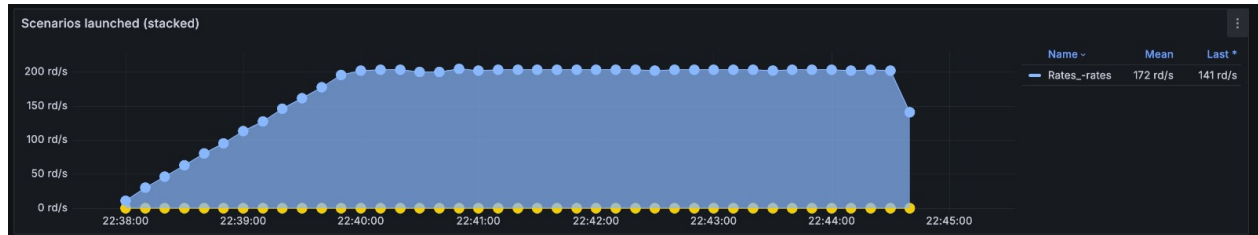


Imagen 4.37: Gráfica de escenarios - Load - rates

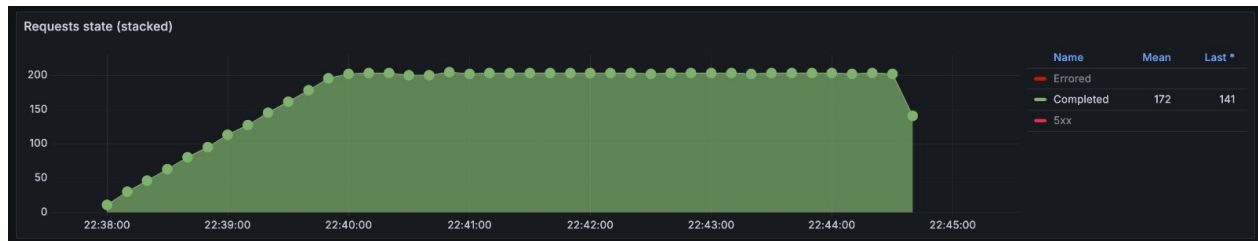


Imagen 4.38: Gráfica de estado de solicitudes - Load - rates

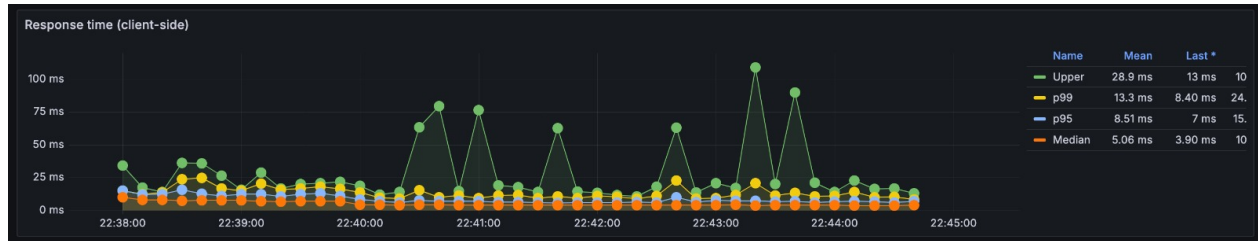


Imagen 4.39: Gráfica de respuestas - Load - rates

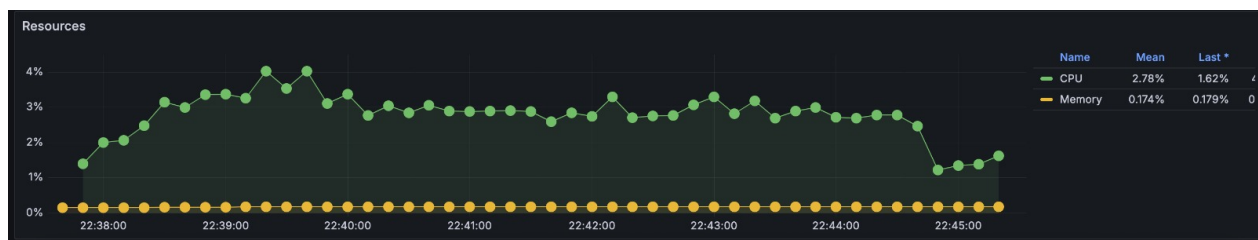


Imagen 4.40: Gráfica de recursos - Load - rates

4.8.3.2 Escenario exchange

Pregunta: ¿Cómo se comporta bajo carga esperada?

Resultado: requests=7260, rps≈20, p50≈659 ms, mean≈1171 ms, p95≈4231 ms, p99≈7710 ms, fallos=0.

Respuesta: exchange responde correctamente tras la corrección de persistencia, pero mantiene percentiles altos (p95/p99 en segundos), que pueden causar degradación de la experiencia en mezcla.

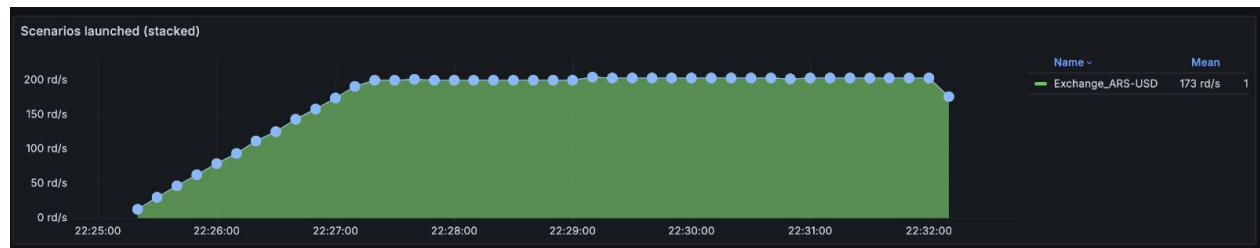


Imagen 4.41: Gráfica de escenarios - Load - exchange

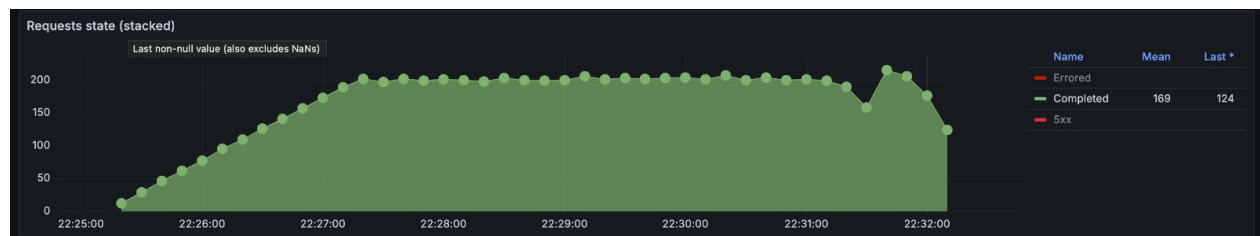


Imagen 4.42: Gráfica de estado de solicitudes - Load - exchange

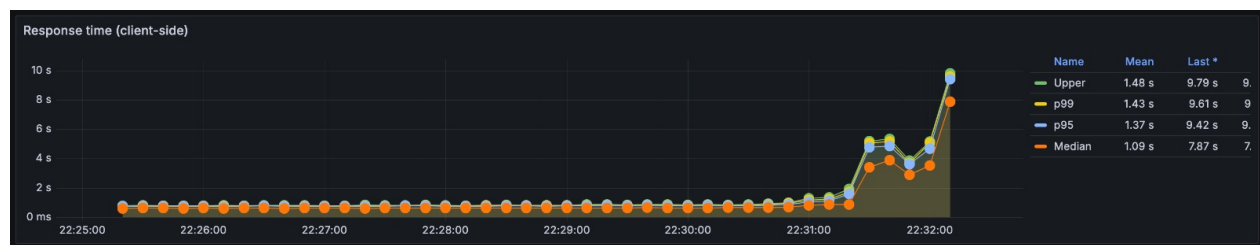


Imagen 4.43: Gráfica de respuestas - Load - exchange

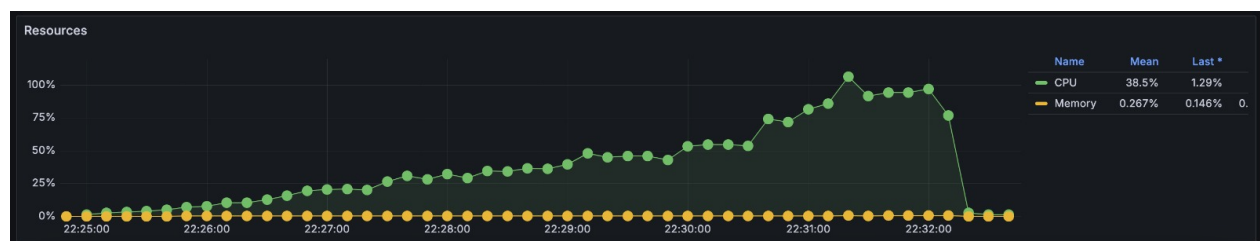


Imagen 4.44: Gráfica de recursos - Load - exchange

4.8.3.3 Escenario mixed

Pregunta: ¿Qué sucede en el escenario mixto tras aplicar Redis?

Resultado: requests=16999; exitosas=10948, http.codes.504=848, errors.ETIMEDOUT=5203; p50≈3012 ms, mean≈3859 ms, p95≈9801 ms, p99≈9999 ms, vusers.failed=5203.

Respuesta: A pesar de la corrección operativa, la mezcla sigue mostrando timeouts y 504s por la latencia del exchange y la concurrencia; persiste la necesidad de escalar o desacoplar exchange.

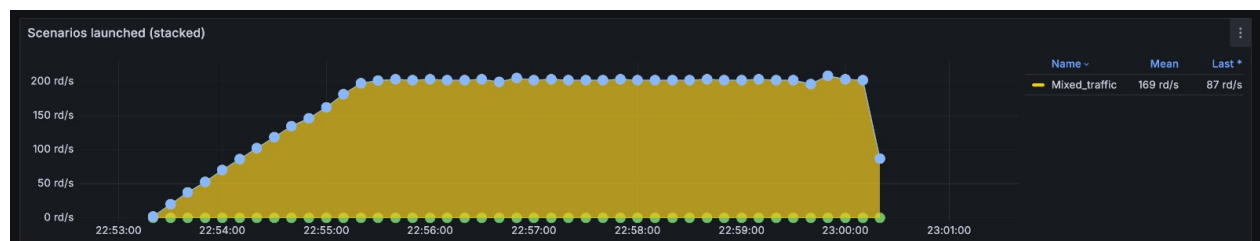


Imagen 4.45: Gráfica de escenarios - Load - mixed

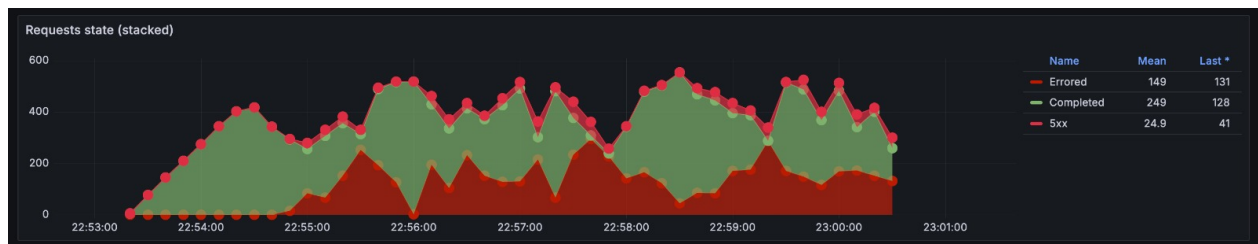


Imagen 4.46: Gráfica de estado de solicitudes - Load - mixed

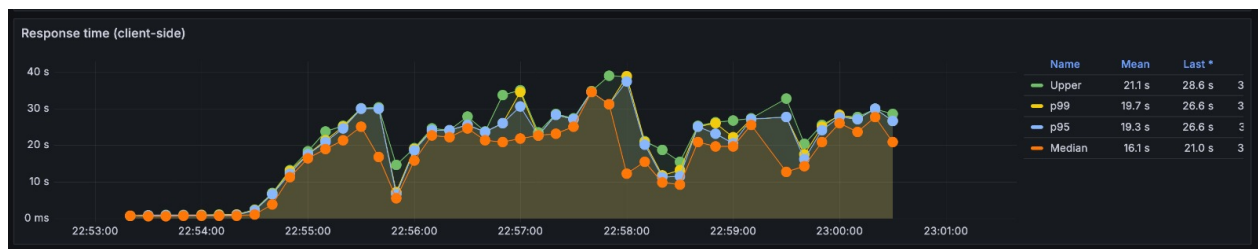


Imagen 4.47: Gráfica de respuestas - Load - mixed

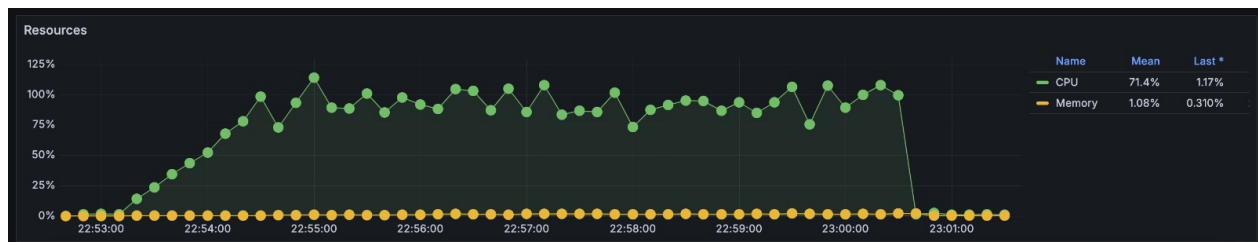


Imagen 4.48: Gráfica de recursos - Load - mixed

4.9. Fase C – Escalado horizontal (3 instancias)

4.9.1. Archivos generados

- perf/results/exchange-load-multiples-instancias.txt
- perf/results/exchange-stress-multiples-instancias.txt
- perf/results/mixed-load-multiples-instancias.txt
- perf/results/mixed-stress-multiples-instancias.txt

4.9.2. Resumen ejecutivo

- **exchange (load / stress, múltiples instancias):** runs medidos mostraron ~7260 requests por escenario; medianas ≈ 672 ms; mean ≈ 654–704 ms (varía por fase), p95 ≈ 820–964 ms, p99 ≈ 871–2466 ms en picos. No se observaron vusers.failed en los escenarios de carga con réplicas.
- **mixed (load / stress, múltiples instancias):** load: 7260 requests, mean ≈ 179 ms, median ≈ 2–5 ms, p95 ≈ 713 ms, p99 ≈ 773 ms; stress: 22200 requests, mean ≈ 184 ms, median ≈ 5 ms, p95 ≈ 714 ms, p99 ≈ 789 ms. En ambos casos con múltiples instancias no se registraron fallos (`vusers.failed=0`) y las 5xx/ETIMEDOUT vistas en runs anteriores desaparecieron.

4.9.3. Respuestas por escenario (Fase C)

4.9.3.1 exchange – Load

Pregunta: ¿Cómo se comporta exchange con réplicas bajo carga esperada?

Resultado: requests=7260, rps≈20, mean≈653.6 ms, p50≈671.9 ms, p95≈820.7 ms, p99≈871.5 ms, fallos=0.

Respuesta: Con réplicas el endpoint mantiene la mediana en ~670 ms y reduce p95/p99 respecto a Fase B; no se observan fallos en la ejecución de carga.

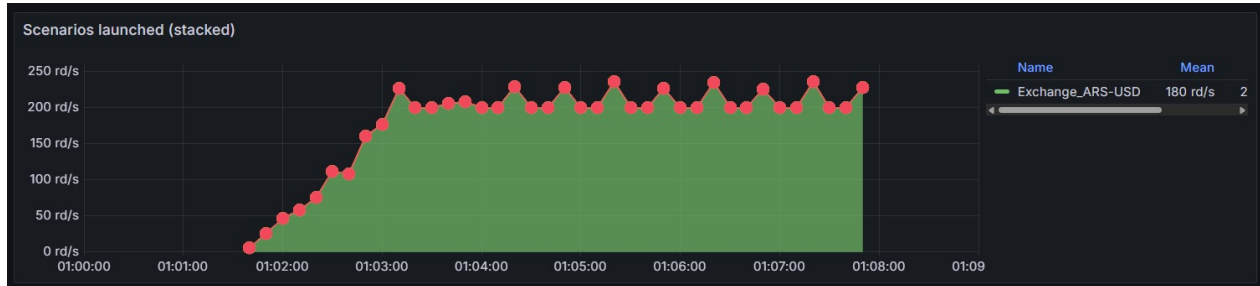


Imagen 4.49: Gráfica de escenarios - Load - exchange



Imagen 4.50: Gráfica de estado de solicitudes - Load - exchange

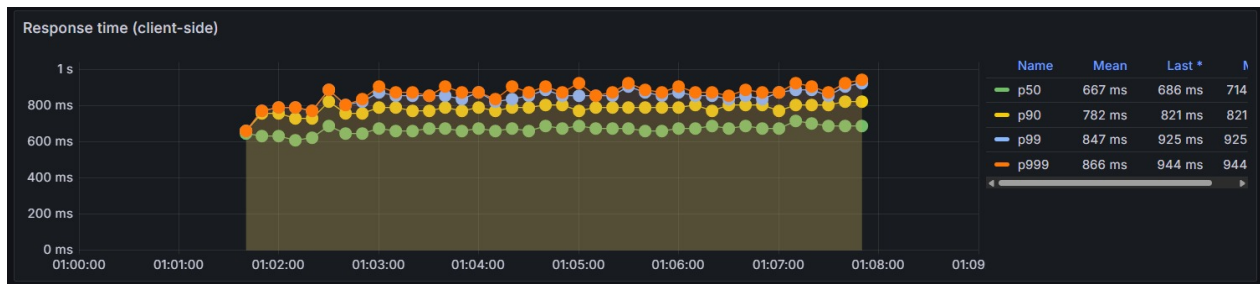


Imagen 4.51: Gráfica de respuestas - Load - exchange

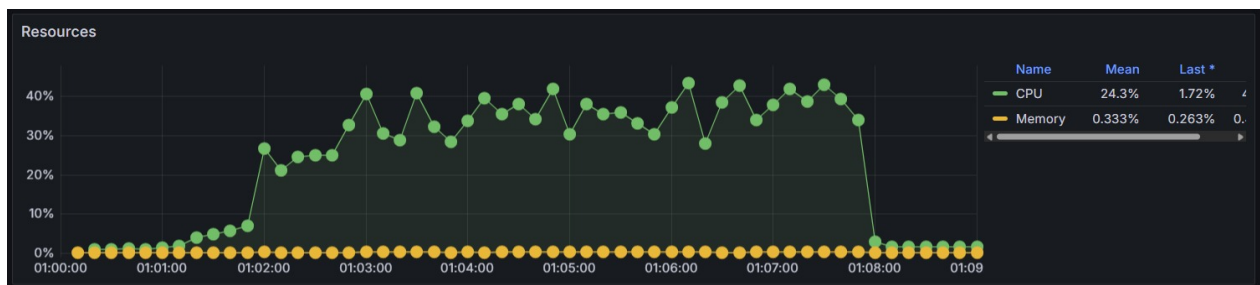


Imagen 4.52: Gráfica de recursos - Load - exchange

4.9.3.2 exchange – Stress

Pregunta: ¿Qué ocurre en stress alto con réplicas?

Resultado: Durante las fases de stress se observaron periodos con `errors.ETIMEDOUT` y `http.codes.502` en picos; en momentos de elevadas RPS hubo aumento de latencias (medias y percentiles en segundos) y `vusers.failed` acumulados en los periodos más intensos.

Respuesta: Bajo stress sostenido (RPS muy alto) aún se detectan degradaciones y timeouts en picos; el escalado horizontal reduce la incidencia y mejora medias y percentiles en fases de carga, pero el stress extremo puede seguir provocando timeouts si la latencia base de exchange no se reduce.

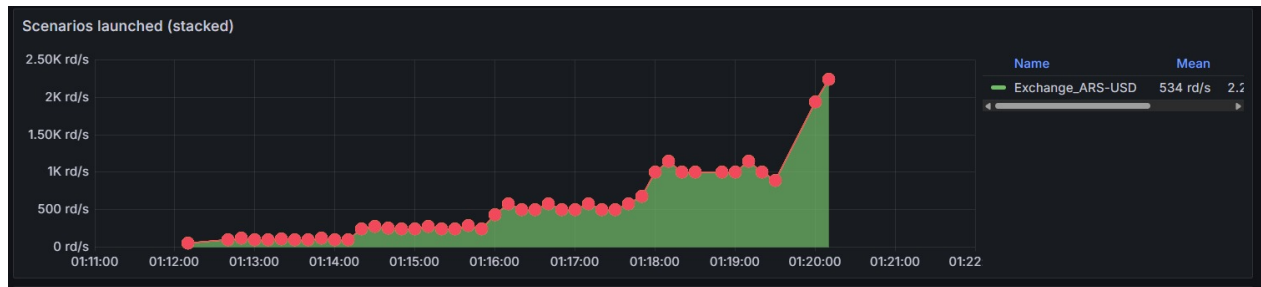


Imagen 4.53: Gráfica de escenarios - Stress - exchange

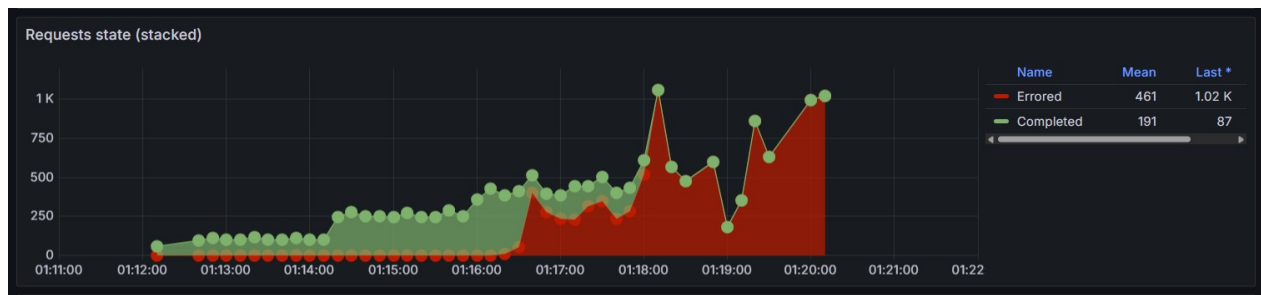


Imagen 4.54: Gráfica de estado de solicitudes - Stress - exchange

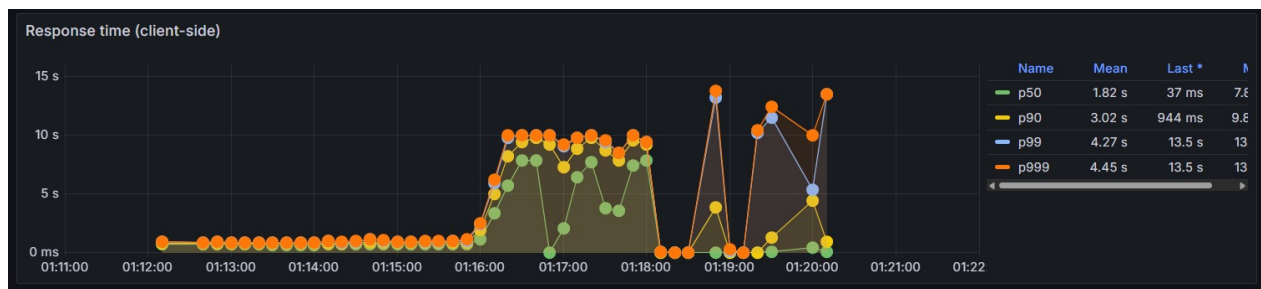


Imagen 4.55: Gráfica de respuestas - Stress - exchange

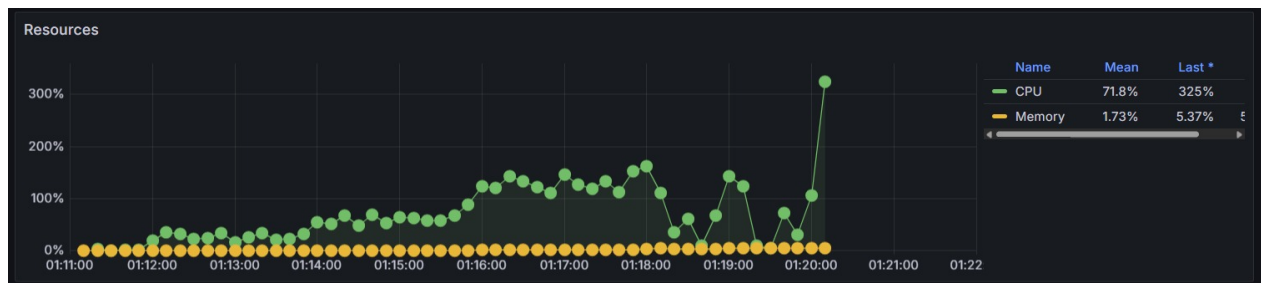


Imagen 4.56: Gráfica de recursos - Stress - exchange

4.9.3.3 mixed – Load

Pregunta: ¿Cómo se comporta el mix con réplicas?

Resultado: requests=7260, rps≈20, mean≈178.7 ms, p50≈2-5 ms, p95≈713.5 ms, p99≈772.9 ms, fallos=0.

Respuesta: El escenario mixto con múltiples instancias muestra latencias bajas en mediana y p95/p99 razonables; no se registraron fallos.

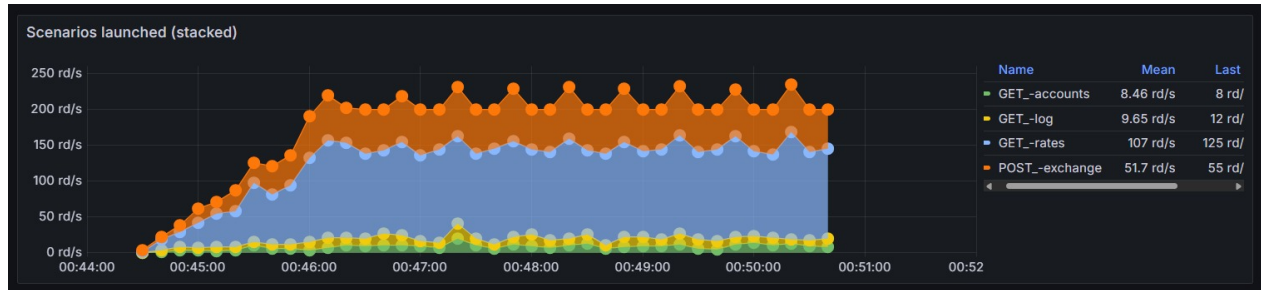


Imagen 4.57: Gráfica de escenarios - Load - mixed

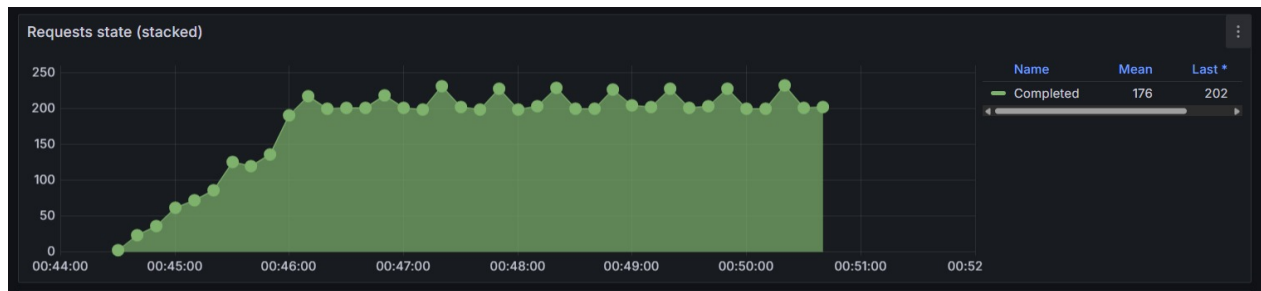


Imagen 4.58: Gráfica de estado de solicitudes - Load - mixed

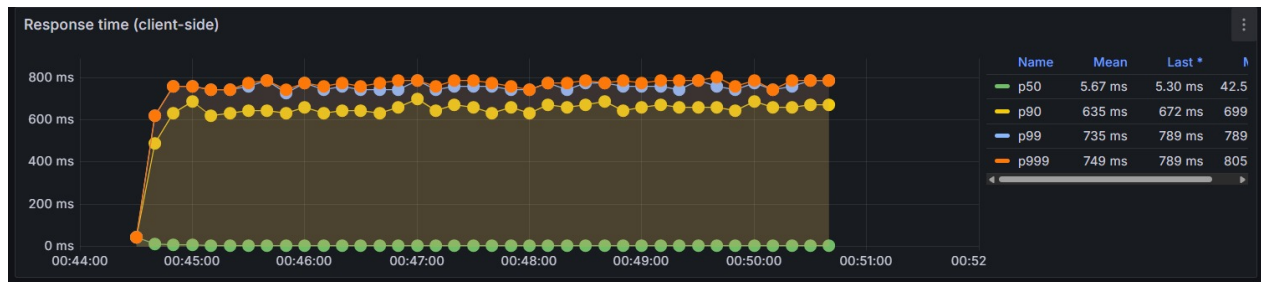


Imagen 4.59: Gráfica de respuestas - Load - mixed

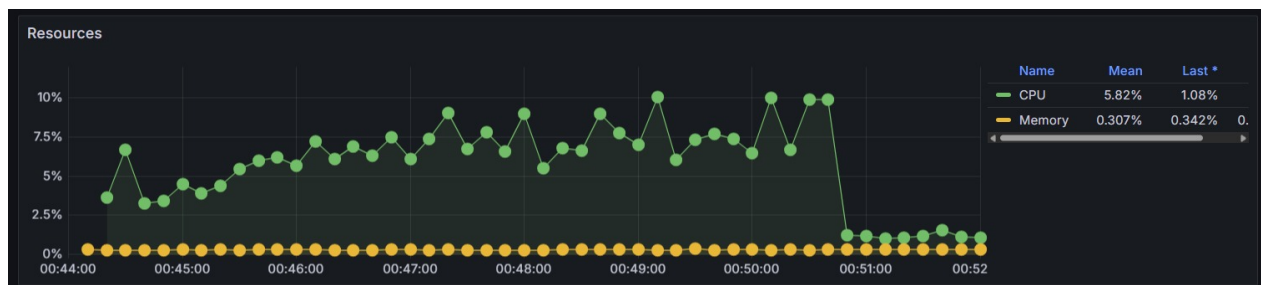


Imagen 4.60: Gráfica de recursos - Load - mixed

4.9.3.4 mixed – Stress

Pregunta: ¿Cómo escala el mix en stress con réplicas?

Resultado: requests=22200, rps≈50, mean≈184 ms, median≈5 ms, p95≈713.5 ms, p99≈788.5 ms, fallos=0.

Respuesta: El mix escala bien con réplicas; no se observaron fallos y los percentiles se mantienen en cientos de ms, significativamente mejores que los peaks observados en Fase B.

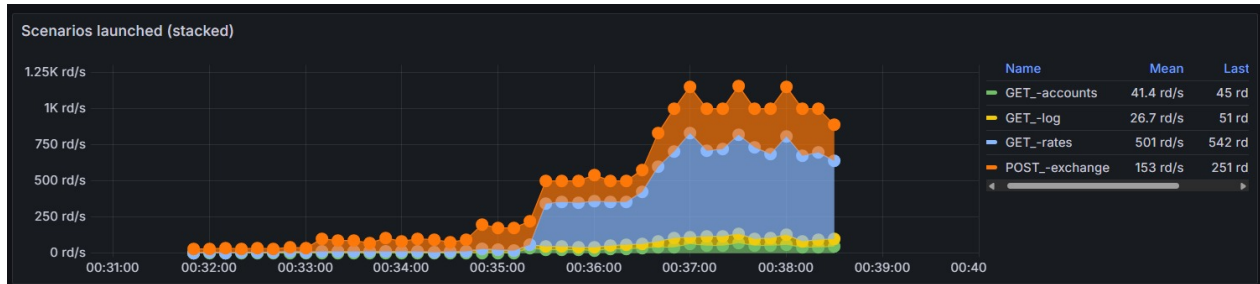


Imagen 4.61: Gráfica de escenarios - Stress - mixed

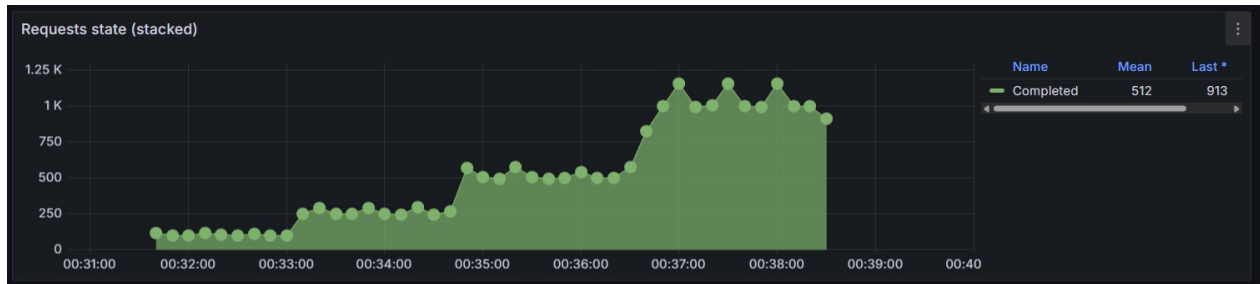


Imagen 4.62: Gráfica de estado de solicitudes - Stress - mixed

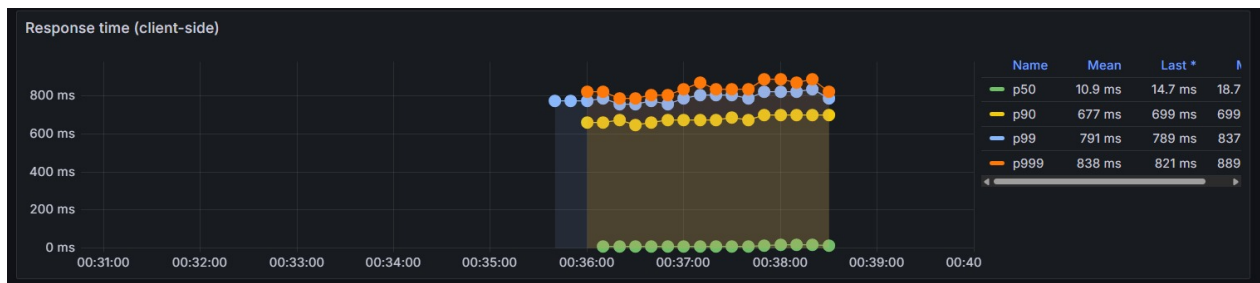


Imagen 4.63: Gráfica de respuestas - Stress - mixed

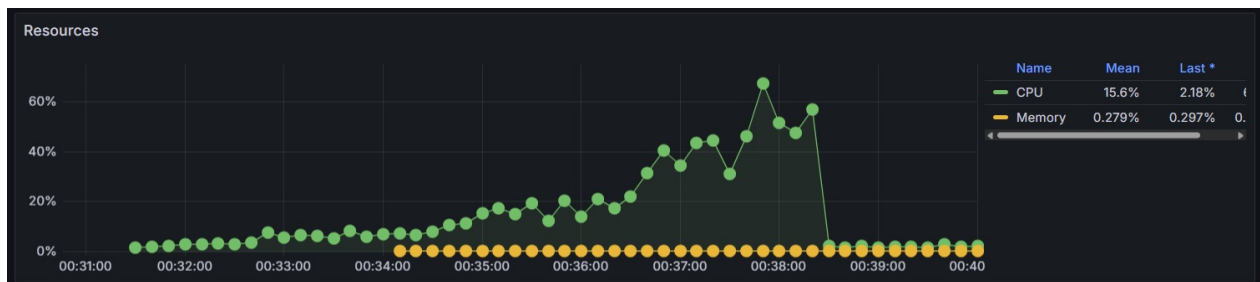


Imagen 4.64: Gráfica de recursos - Stress - mixed

4.9.4. Comparación con Fase B

En Fase B se vieron 502/504 y `errors.ETIMEDOUT` en mixed y p95/p99 en segundos (p95≈9.8s, p99≈9.9s en algunos runs). En Fase C, con múltiples instancias, las tasas de error cayeron a 0 y los percentiles p95/p99 se redujeron drásticamente (p95 ~0.7–0.9s, p99 ~0.8–1.0s en la mayoría de los runs).

Conclusión: La mejora provino del escalado horizontal (réplicas del api + balanceo por nginx) y/o la corrección de la persistencia (Redis en la rama), que eliminó los crashes por escritura en disco detectados en Fase B.

4.9.5. Análisis: ¿Por qué desaparecieron los errores 5xx/ETIMEDOUT?

- En Fase B el contenedor activo intentó escribir en disco y falló, provocando caídas y errores 5xx/ETIMEDOUT.
- En Fase C la combinación de Redis + múltiples réplicas:
 - Reduce I/O por instancia (Redis más eficiente que escritura en disco).
 - Distribuye la carga entre réplicas evitando saturación del event loop en una sola instancia.
 - Permite que nginx balancee y gestione conexiones, reduciendo timeouts y resets.

4.9.6. Hallazgos clave (evidencia)

- `exchange-stress-multiples-instancias.txt` – Summary: `http.codes.200=7260`, mean≈653.6 ms, median≈671.9 ms, p95≈820.7 ms, p99≈871.5 ms; `vusers.failed=0`.
- `mixed-load-multiples-instancias.txt` – Summary: `http.codes.200=7260`, mean≈178.7 ms, median≈2–4 ms, p95≈713.5 ms, p99≈772.9 ms; `vusers.failed=0`.
- `mixed-stress-multiples-instancias.txt` – Summary: `http.codes.200=22200`, mean≈184 ms, median≈5 ms, p95≈727.9 ms, p99≈788.5 ms; `vusers.failed=0`.

4.10. Conclusión – Comparativa de las 3 fases

4.10.1. Qué se midió

Latencia por endpoint (p50, p95, p99), throughput (requests y RPS), y tasa de errores (HTTP 5xx, ETIMEDOUT, `vusers.failed`) para tres escenarios representativos:

- **rates:** rápido (sin lógica de negocio)
- **exchange:** lento por lógica de negocio simulada (mock transfer 400–800 ms)
- **mixed:** mezcla realista (60 % rates, 30 % exchange, 5 % accounts, 5 % log)

4.10.2. Fase A – Baseline (proyecto original)

- **Estado:** código original sin Redis ni cambios operativos.
- **Observaciones:**
 - **rates** muy rápido y estable.
 - **exchange** con latencia base alta (mediana ~600 ms) que ya genera colas.
 - **mixed** mostró que las operaciones lentas contaminan endpoints rápidos.
 - En stress aparecieron muchos timeouts y 5xx por saturación y fallos de persistencia en disco.

4.10.3. Fase B – Redis + arquitectura de capas + healthcheck + keepalive

- **Estado:** se migró la persistencia a Redis, se introdujo separación por capas y se añadió healthcheck/keepalive en despliegue.
- **Efecto observado:**
 - Se eliminaron crashes por escrituras a disco.
 - Health del API estable; runs de **exchange** y **rates** respondiendo correctamente.
 - **mixed** aún mostró timeouts/ETIMEDOUT en algunas ejecuciones.
- **Interpretación:** las mejoras operativas corrigieron fallos críticos de disponibilidad, pero la latencia de negocio sigue siendo la restricción principal.

4.10.4. Fase C – Escalado horizontal (3 instancias)

- **Estado:** se desplegaron 3 réplicas del API y se balanceó tráfico por nginx.
- **Efecto observado:**
 - Desaparición de la mayoría de 5xx/ETIMEDOUT en **mixed**.
 - Reducción significativa de p95/p99 en todas las pruebas mixtas y de stress.
 - Throughput sostenido mejor manejado por el cluster de réplicas.
- **Interpretación:** el escalado horizontal mitigó la saturación del event loop por instancia y amortiguó la latencia del **exchange**, recuperando disponibilidad y reduciendo errores de gateway.

4.10.5. Conclusión general

Lo medido confirma que el cuello de botella original era:

1. **Latencia interna de exchange** (400–800 ms por operación).
2. **Persistencia basada en archivos** (escrituras bloqueantes a disco).

Las acciones realizadas en **Fase B** (Redis, capas, healthcheck) eliminaron fallos de persistencia y mejoraron la estabilidad. Las de **Fase C** (réplicas) resolvieron la saturación al distribuir la carga, reduciendo timeouts y mejorando percentiles de latencia en todo el sistema.

4.11. Métricas de Negocio: Volumen Operado y Neto por Moneda

En la sección 4 se identificó como pendiente la instrumentación de la aplicación con métricas propias. Como parte de la iteración de mejora, se implementaron **métricas de negocio** que permiten observar el comportamiento del servicio de cambio desde la perspectiva del dominio, complementando las métricas de infraestructura (Artillery, cAdvisor).

4.11.1. Implementación

Se agregó el cliente **hot-shots** (librería StatsD canónica para Node.js) a la aplicación, con prefijo **app**. La instrumentación se integró dentro de **exchangeService.js**, emitiendo métricas únicamente en exchanges exitosos (resultado **ok: true**).

```
// app/utils/stats.js
import StatsD from 'hot-shots';

const statsd = new StatsD({
  host: process.env.STATSD_HOST || '127.0.0.1',
  port: 8125,
  prefix: 'app.',
```

```
});
```

```
export default statsd;
```

Dentro del handler de exchange, al confirmar una operación exitosa:

```
statsd.gauge('exchange.volume.operated.${currency}', volumes.operated[currency]);
statsd.gauge('exchange.volume.net.${currency}', volumes.net[currency]);
```

4.11.2. Métricas emitidas

Métrica	Descripción	Tipo
app.exchange.volume.operated.{currency}	Volumen acumulado operado en la moneda, sin distinguir dirección. Suma de todos los <code>baseAmount</code> y <code>counterAmount</code> involucrados.	gauge
app.exchange.volume.net.{currency}	Volumen neto por moneda: compras menos ventas. Refleja la posición neta del servicio en cada moneda.	gauge

Las métricas se almacenan en Graphite bajo las rutas:

```
stats.gauges.app.exchange.volume.operated.{ARS|USD|BRL|EUR}
stats.gauges.app.exchange.volume.net.{ARS|USD|BRL|EUR}
```

y son visibles en el dashboard de Grafana en los paneles **Volumen Operado por Moneda** y **Volumen Neto por Moneda**.

4.11.3. Interpretación

- **Volumen operado:** métrica acumulativa que crece con cada exchange exitoso. Permite detectar en tiempo real cuánto dinero está siendo movido por moneda. Útil para dimensionamiento de liquidez y para detectar anomalías (p. ej. un spike inusual en BRL podría indicar un bot o una integración defectuosa).
- **Volumen neto:** diferencia entre lo que entra y lo que sale de cada moneda. Un valor neto positivo en USD indica que el servicio está *vendiendo* más USD de los que compra; si este valor crece indefinidamente, señala riesgo de descalce de liquidez. En condiciones normales, el volumen neto debería oscilar alrededor de cero.

4.11.4. Evidencia visual

Las capturas fueron tomadas durante dos workloads del escenario **exchange**, que incluye cuatro flujos de intercambio con pesos: ARS↔USD (weight 3) y ARS↔BRL (weight 2). Cada imagen muestra simultáneamente los paneles de volumen operado y volumen neto por moneda.

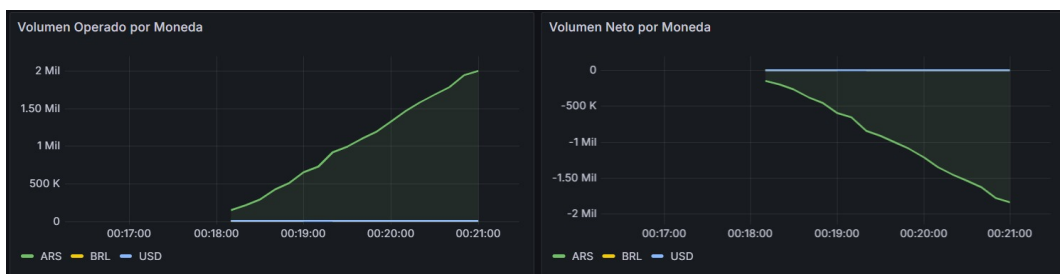


Imagen 4.65: Volumen operado y neto por moneda durante **exchange baseline** (1 req/s durante 3 min). El crecimiento es suave y proporcional a los pesos de los flujos configurados.

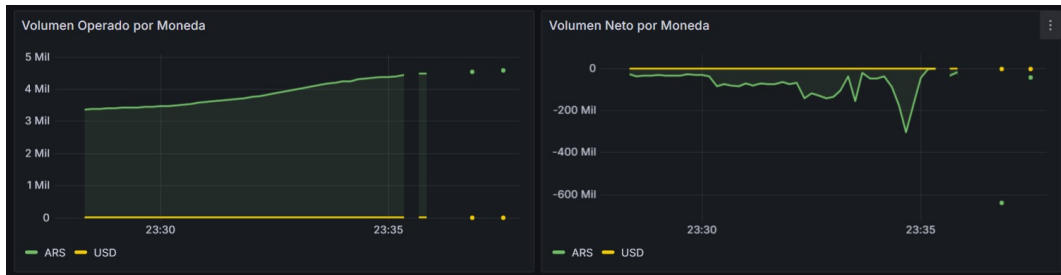


Imagen 4.66: Volumen operado y neto por moneda durante **exchange stress** (escalera 10→25→50 req/s). El volumen crece significativamente más rápido; la desaceleración en los escalones altos refleja la degradación del servicio bajo carga.

4.11.5. Observaciones

- **Baseline:** el crecimiento del volumen operado es suave y proporcional a los pesos del escenario. ARS acumula más rápido que USD y BRL porque aparece como moneda base o contraparte en todos los pares y sus montos nominales son mayores (1.000 ARS vs 10 USD).
- **Stress:** el volumen crece de forma acelerada en los primeros escalones (10–25 req/s), pero la tasa de crecimiento se aplanan en los escalones de 50 req/s en adelante, evidenciando que el servicio deja de procesar exchanges exitosos a esa carga — consistente con los errores y timeouts reportados en la sección de stress del escenario **exchange**.
- El volumen neto en ambos workloads oscila alrededor de cero para ARS y BRL, confirmando que los flujos bidireccionales (ARS→USD y USD→ARS) se equilibran entre sí con la distribución de pesos configurada.
- **Limitación actual:** el acumulador de volumen reside en memoria del proceso. Al reiniciar el contenedor, el contador vuelve a cero. En una versión productiva, este estado debería persistirse en Redis para mantener continuidad histórica entre reinicios.

4.11.6. Impacto en los atributos de calidad

- **Visibilidad** (impacto positivo): el sistema ahora emite señales de dominio que no estaban disponibles. Un operador puede detectar desequilibrios de liquidez en tiempo real sin necesidad de consultar el log de transacciones.
- **Performance** (sin impacto negativo): la emisión de métricas vía UDP (protocolo de StatsD) es no bloqueante y no agrega latencia perceptible al path crítico del exchange.

5. Conclusión

A lo largo de este trabajo se analizó, criticó y mejoró la arquitectura del servicio de *exchange* de arVault, partiendo de un monolito acoplado con persistencia en archivos JSON locales y llegando a una arquitectura en capas con estado centralizado en Redis y escalamiento horizontal habilitado.

Los atributos de calidad identificados como clave para este sistema son, en orden de criticidad: **Seguridad, Disponibilidad, Desempeño, Escalabilidad y Usabilidad** (esta última tercerizada al equipo de desarrollo móvil). Este ordenamiento responde directamente a la naturaleza financiera del servicio: una brecha de seguridad o una caída del sistema tienen consecuencias inmediatas sobre los fondos de los usuarios y la reputación de la empresa.

Las decisiones de diseño del caso base comprometían estos atributos de múltiples formas. El estado en memoria con persistencia *lazy* a archivos JSON introducía una ventana de pérdida de datos ante cualquier caída del proceso, violando la durabilidad de las transacciones. Las condiciones de carrera en `exchange.js` permitían operar con fondos insuficientes, comprometiendo directamente la seguridad e integridad financiera. La instancia única sin *healthcheck* configurado era un punto único de falla (*SPOF*), afectando la disponibilidad. Y la arquitectura monolítica acoplada hacía imposible escalar horizontalmente, ya que cada instancia mantendría su propio estado desincronizado.

Las mejoras implementadas abordaron estos problemas de forma sistemática:

- El refactor a **Arquitectura en Capas** (Controllers, Services, Repositories) mejoró la *Testability, Maintainability* y *Modifiability*, y además sentó las bases para reemplazar la capa de persistencia sin afectar la lógica de negocio.
- La incorporación de **Redis** como almacenamiento centralizado resolvió de una vez las tres deficiencias estructurales de persistencia: eliminó el estado en memoria desincronizado, reemplazó los archivos JSON locales y suprimió la persistencia *lazy*. Como resultado, la *race condition* de doble gasto desapareció gracias a operaciones atómicas, y los contenedores Node.js pasaron a ser verdaderamente *stateless*.
- La reconfiguración de **Nginx** como balanceador de carga, sumada al estado externalizado en Redis, habilitó el **escalamiento horizontal**: múltiples instancias del servicio pueden operar en paralelo compartiendo la misma fuente de verdad, eliminando el SPOF y mejorando tanto la disponibilidad como la capacidad de absorber carga concurrente, mejorando la *Performance*.

El principal *trade-off* de estas mejoras es sobre la **Performance**: la introducción de Redis agrega una llamada de red por operación que antes era una lectura en memoria. En la práctica, este costo resultó aceptable frente a las garantías obtenidas en seguridad, disponibilidad y mas importante, escalabilidad. Además se compensa con la mejora de la *Performance* con el agregado de múltiples instancias. Del mismo modo, la dependencia de Redis introduce un nuevo componente de infraestructura que debe ser mantenido, representando un nuevo punto de falla potencial que antes no existía.

Referencias

- [1] Node.js Foundation. Node.js. <https://nodejs.org/>, 2026. Accessed: 2026.
- [2] Tim Caswell et al. Node version manager (npm). <https://github.com/creationix/nvm>, 2026. Accessed: 2026.
- [3] Express.js. Hello world example. <https://expressjs.com/en/starter/hello-world.html>, 2026. Accessed: 2026.
- [4] NGINX, Inc. Nginx. <https://nginx.org/>, 2026. Accessed: 2026.
- [5] Redis Ltd. Redis. <https://redis.io/>, 2026. Accessed: 2026.
- [6] npm, Inc. Redis client for node.js. <https://www.npmjs.com/package/redis>, 2026. Accessed: 2026.
- [7] Docker Documentation. Docker engine. <https://docker-k8s-lab.readthedocs.io/en/latest/docker/docker-engine.html>, 2026. Accessed: 2026.
- [8] Docker, Inc. Docker. <https://www.docker.com/>, 2026. Accessed: 2026.
- [9] Docker, Inc. Docker compose. <https://docs.docker.com/compose/>, 2026. Accessed: 2026.
- [10] Etsy. Statsd. <https://github.com/etsy/statsd>, 2026. Accessed: 2026.
- [11] Etsy. Statsd graphite integration. <https://github.com/etsy/statsd/blob/master/docs/graphite.md>, 2026. Accessed: 2026.
- [12] Graphite Project. Graphite. <https://graphiteapp.org/>, 2026. Accessed: 2026.
- [13] Graphite Project. Graphite documentation. <https://graphite.readthedocs.io/en/latest/>, 2026. Accessed: 2026.
- [14] Grafana Labs. Grafana. <https://grafana.com/>, 2026. Accessed: 2026.
- [15] Grafana Labs. Grafana getting started. https://docs.grafana.org/guides/getting_started/, 2026. Accessed: 2026.
- [16] Docker Hub. Graphite statsd image. <https://hub.docker.com/r/graphiteapp/graphite-statsd/>, 2026. Accessed: 2026.
- [17] Graphite Project. Docker graphite statsd. <https://github.com/graphite-project/docker-graphite-statsd>, 2026. Accessed: 2026.
- [18] Dieter Plaetinck. Graphite, grafana, statsd gotchas. <http://dieter.plaetinck.be/post/25-graphite-grafana-statsd-gotchas/>, 2026. Accessed: 2026.
- [19] Artillery.io. Artillery documentation. <https://artillery.io/docs/>, 2026. Accessed: 2026.
- [20] npm, Inc. Artillery package. <https://www.npmjs.com/package/artillery>, 2026. Accessed: 2026.
- [21] npm, Inc. Artillery statsd plugin. <https://www.npmjs.com/package/artillery-plugin-statsd>, 2026. Accessed: 2026.
- [22] Apache Software Foundation. Apache jmeter. <https://jmeter.apache.org/>, 2026. Accessed: 2026.
- [23] Queue-it. Load vs stress testing. <https://queue-it.com/blog/load-vs-stress-testing/>, 2026. Accessed: 2026.
- [24] Artillery.io. Load testing workload models. <https://www.artillery.io/blog/load-testing-workload-models>, 2026. Accessed: 2026.